



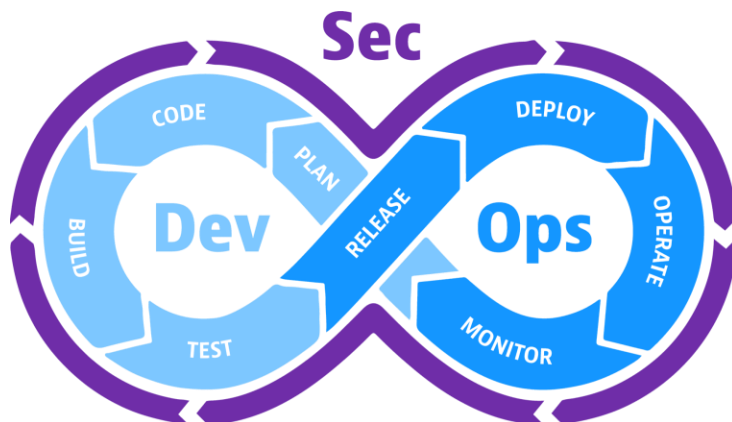
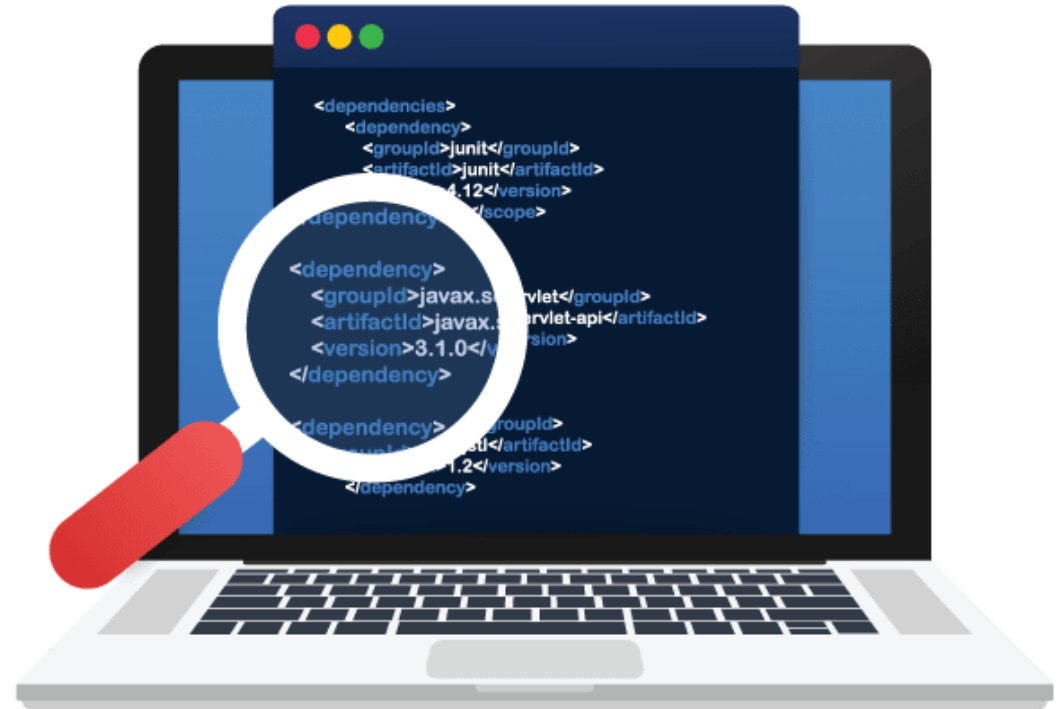
NT521 - Lập trình an toàn & Khai thác lỗ hổng phần mềm

SSDLC - Quy trình phát triển phần mềm an toàn



- Định nghĩa Quy trình phát triển phần mềm an toàn (Secure SDLC – SSDLC)
- Đặc tả yêu cầu an toàn (Security Requirement)
- Thiết kế an toàn (Secure Design)
- Lập trình An toàn (Secure Code)
- Khung phát triển phần mềm an toàn (Security Software Development Framework – SSDF)
- Các lỗi thường gặp (Common Pitfalls)
- Kiểm thử và triển khai an toàn (Secure Testing and Deployment)

Phân tích mã nguồn: SAST vs DAST



Static application security testing (SAST)

Dynamic Application Security Testing (DAST)

Công cụ phân tích mã nguồn

Phân tích mã nguồn tĩnh là gì?

- Phân tích mã nguồn tĩnh, hay Phân tích mã nguồn là phương pháp thực hiện trên mã nguồn **“tĩnh” (không thực thi) của phần mềm** với các công cụ phân tích mã nguồn tĩnh để phát hiện các lỗ hổng tiềm ẩn.
- Các công cụ phân tích mã nguồn tĩnh kiểm tra toàn bộ mã nguồn để tìm các **lỗ hổng nhất định** cũng như **việc tuân thủ các tiêu chuẩn** lập trình khác nhau.



Công cụ phân tích mã nguồn

Vì sao phải phân tích mã nguồn (tĩnh)?

- Xem xét mã nguồn trước khi thực thi.
- Phân tích nhanh hơn so với phân tích động.
- Có thể tự động hóa quản lý chất lượng mã nguồn.
- Có thể tự động hóa việc tìm bug ở giai đoạn đầu.
- Có thể tự động hóa việc tìm các vấn đề bảo mật ở giai đoạn đầu.
- Một số công cụ phân tích mã nguồn tĩnh đã được tích hợp sẵn trong các IDE (ví dụ Pycharm sử dụng pep8).



https://owasp.org/www-community/Source_Code_Analysis_Tools

Công cụ phân tích mã nguồn



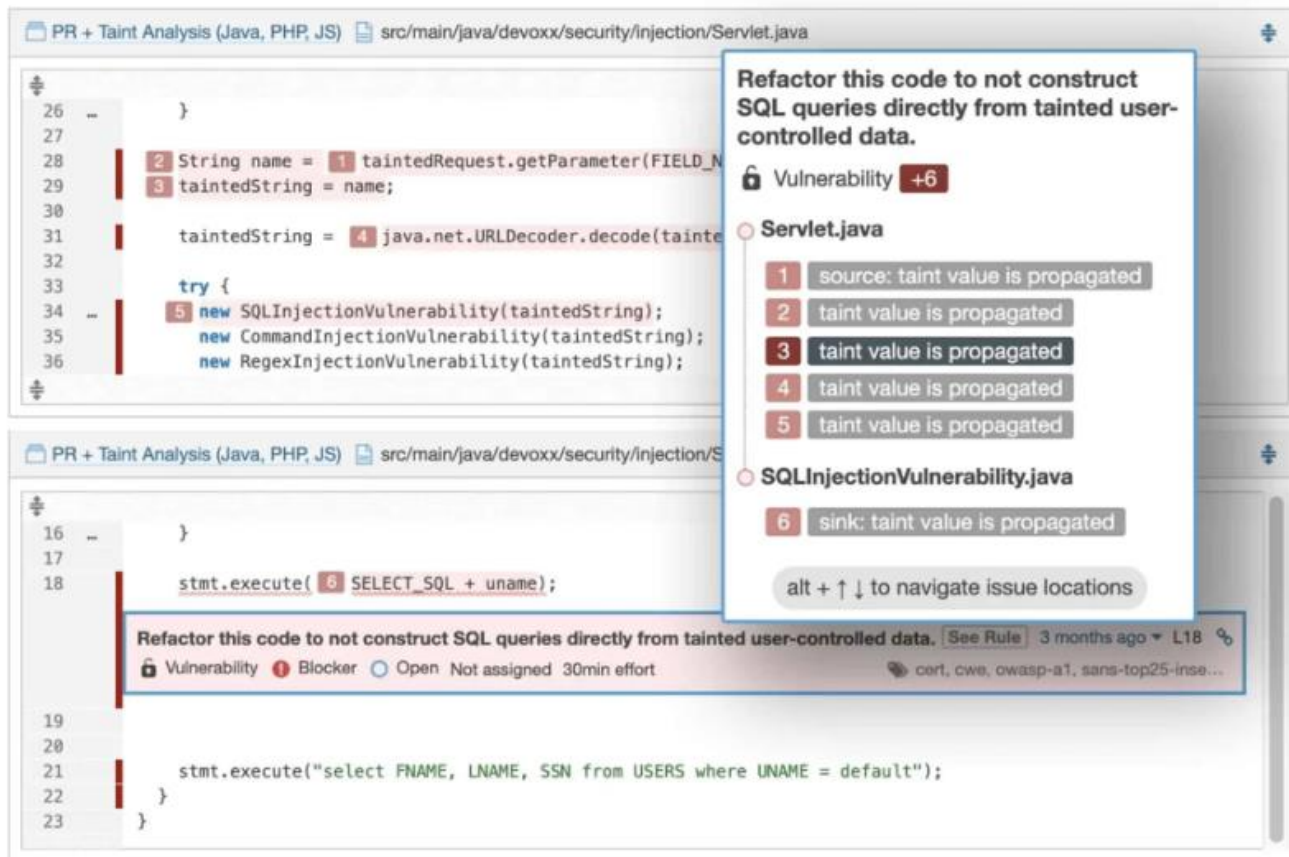
Một số công cụ mã nguồn mở:

- DeepSource
- SonarQube
- Codacy
- DeepScan
- Veracode
- Embold
- Reshift
-

https://owasp.org/www-community/Source_Code_Analysis_Tools

Công cụ phân tích mã nguồn

■ SonarQube



A SonarQube screenshot showing errors

Secure Coding Scanning tool



Tools	Background and key characteristics of the scanning tool
Retire.js	- Detection of vulnerable JavaScript libraries, such as jQuery, AngularJS, Node, and so on. - It provides the command line, grunt plugin, and also OWASP ZAP plugin for integration scanning.
Clang Static Analyzer	This provides standalone command line analysis for C, C++, and Objective C.
Flawfinder	A simple C/C++ code scanning tool. It's a Python command line scanning tool and can be easily customized based on the needs.
DREK	- This acts like GREP to search specific security issue by regular expressions, but it can generate scanning results in PDF or HTML format. - It's easy to extend any scanning rules by regular expressions. It can be used to scan any programming languages.
Pylint	Pylint is a source code checker for the Python programming language.

PHPMD	PHP Mess Detector is a PHP source code scanner.
DawnScanner	Security scanner for Ruby Web applications.
SpotBugs	- This provides a standalone GUI and command line. - SpotBugs can also be used as an Eclipse plugin. It's the successor of FindBugs.
CPP Check	This is a static code analysis tool for C/C++.
Mobile Security Framework (MobSF)	apps. A developer can just upload the APK to the MSF, and the MSF will do all the analysis automatically.
Clang Static Analyzer	This is a code analysis tool for C/C++ and Objective C.
ESLint	- This provides command-line code scanning with JavaScript. - Refer here for the secure code scanning rules: https://eslint.org/docs/rules/.



Secure Coding Scanning tool

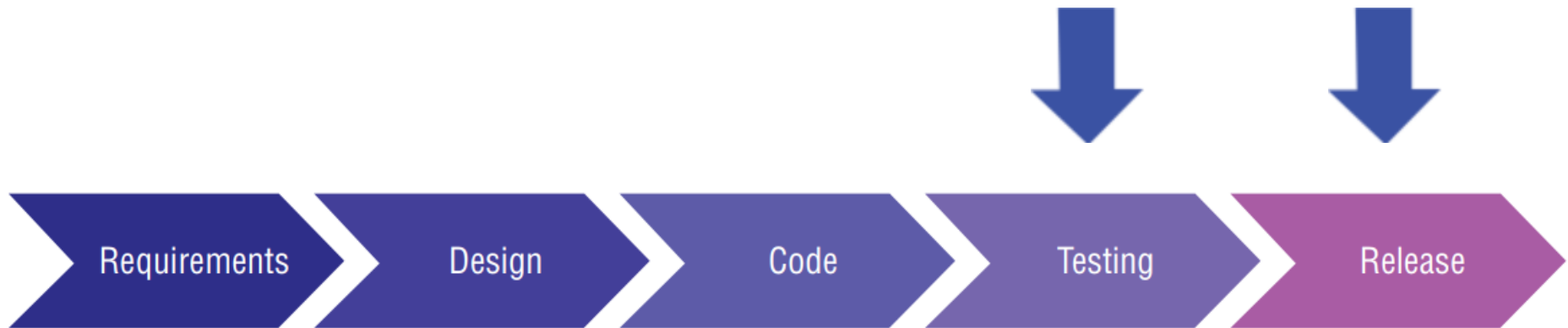


JSHint	This is for JavaScript code scanning, and also provides command line tools by NodeJS.
Infer	This is a static code analyzer for Java, C/C++, and Objective C, provided by Facebook.
Phan	Phan is a static analyzer for PHP.
PHP Security Checker	This checks PHP project dependencies for known security issues.
OWASP Dependency check	This supports a wide range of programming frameworks and checks the disclosed vulnerabilities with updated NVD data feeds. The tool can run as a command line or via integration with Jenkins.
VisualCodeGrepper (VCG)	VCG is a language-independent scanning tool. The scanning rules can also be easily customized by regular expressions. There are also default rules for commonly banned APIs. It provides a GUI and command line to scan any piece of source code.

PMD	This is a source code analyzer for Java and JavaScript. It's mainly for common programming flaws.
Graudit	This is a simple script to find potential security issues by using GREP to search for specific code patterns. The signatures database templates provide clues for what to look for.
SonarQube	This provides support for more than 20 languages and can integrate with CI frameworks. It is also UI-friendly for quality code scanning results.
Brakeman	Static analysis security scanner for Ruby on Rails.
bandit	Security analysis for Python source code.
Error Prone	Error Prone detects potential Java errors during compile time.
Dawn	Dawn is a static analysis security scanner for Ruby web applications.

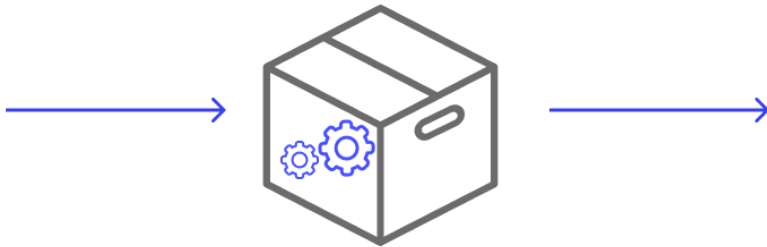


Secure Testing & Deployment

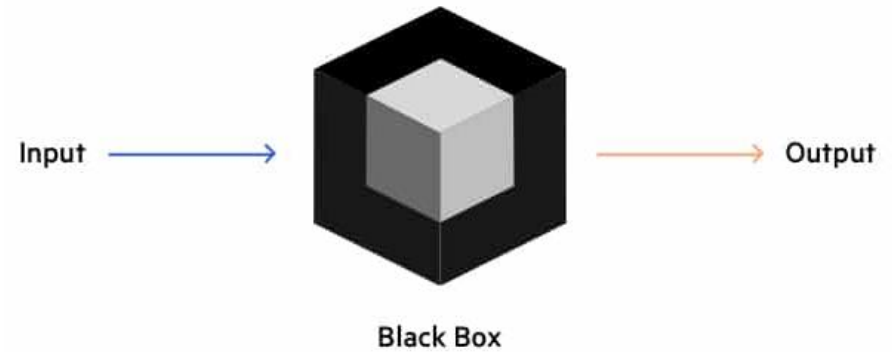




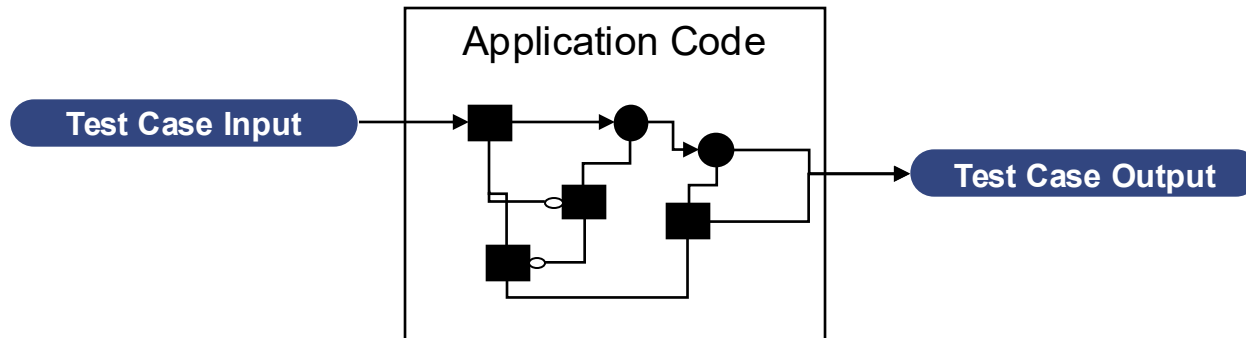
White Box Testing



Black Box Testing



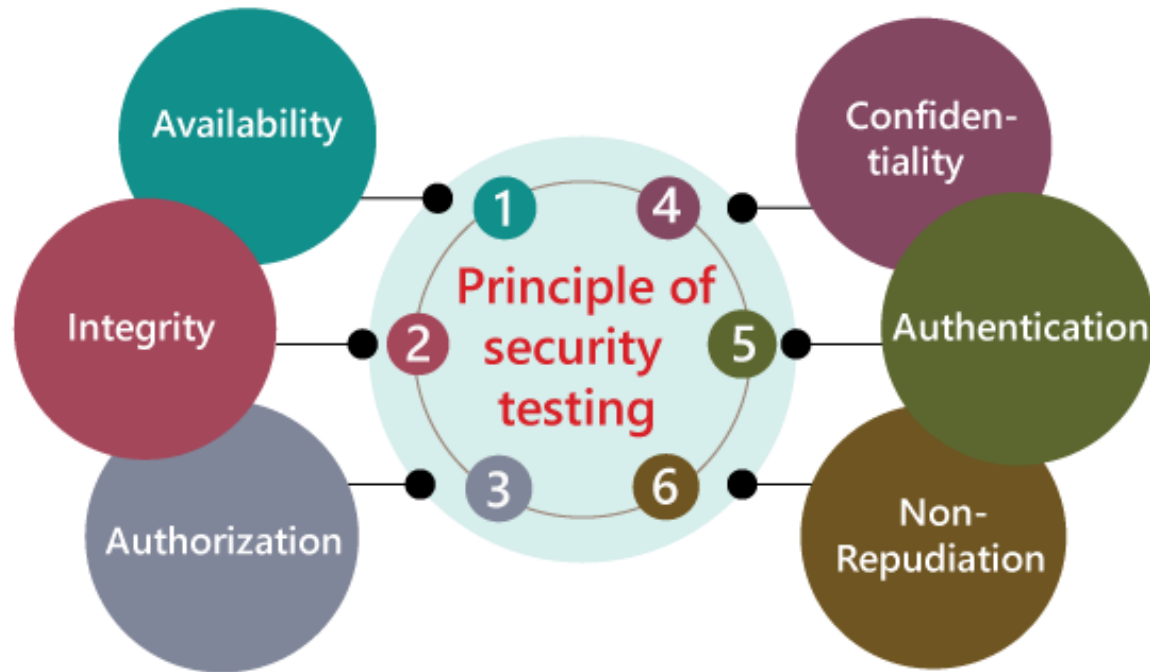
WHITE BOX TESTING APPROACH

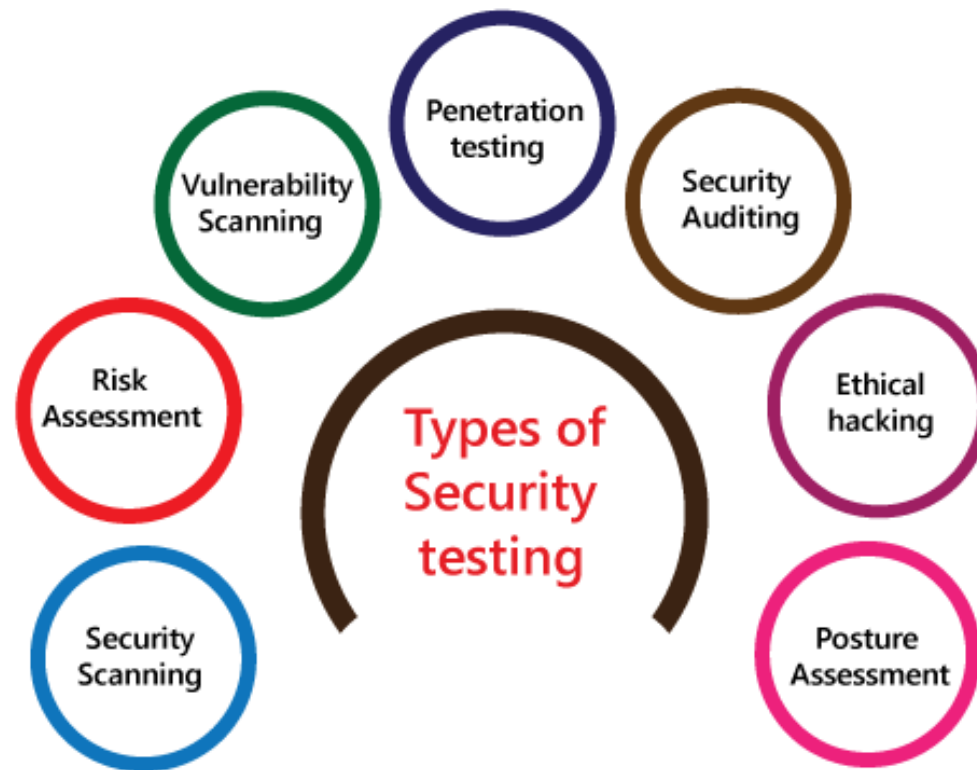




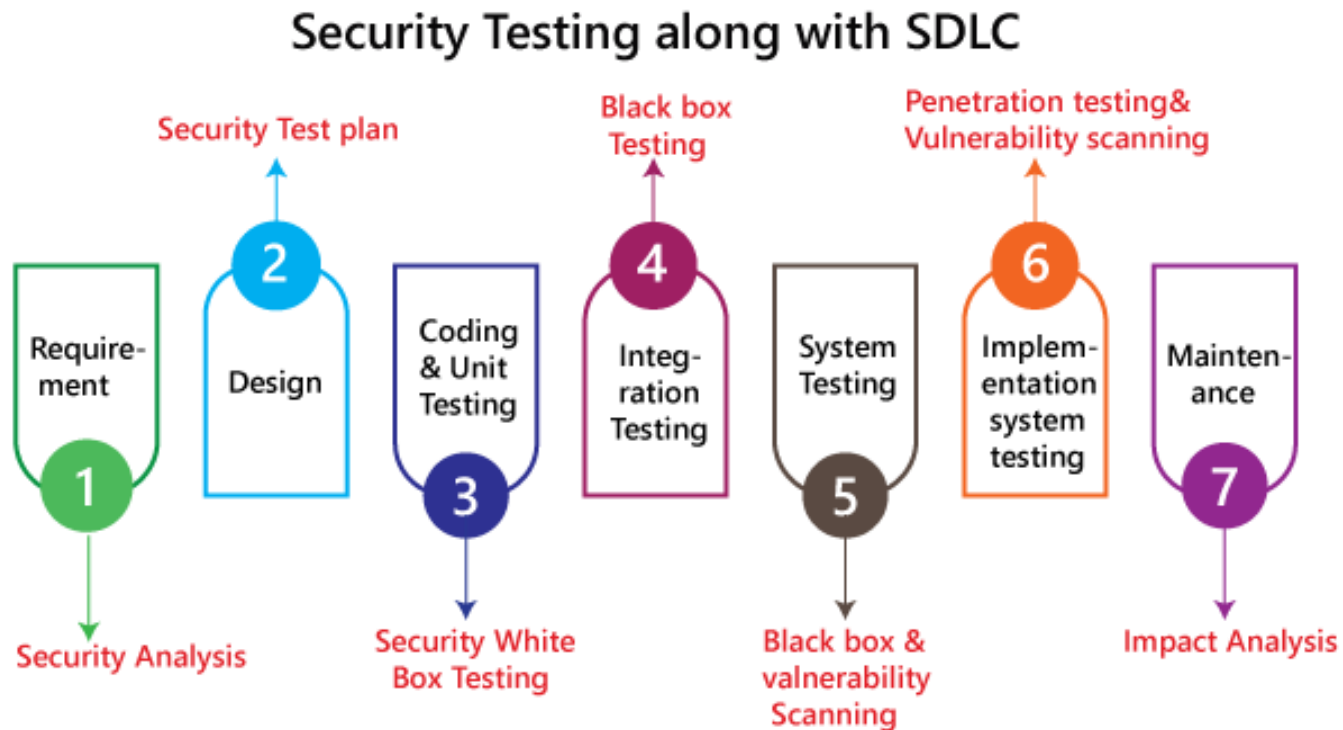
- Kiểm thử mã nguồn
- Kiểm thử ứng dụng
- Kiểm thử hạ tầng
- Kiểm thử cơ sở dữ liệu
- Kiểm thử APIs và các dịch vụ Web
- Kiểm thử tích hợp
- Kiểm thử mạng
- ...Triển khai







Kiểm thử an toàn với SDLC



Kiểm thử an toàn...



Bộ khung phát triển phần mềm an toàn



- Bộ khung phát triển phần mềm An toàn - Secure Software Development Framework (SSDF):
 - Được đề xuất bởi NIST, Hoa Kỳ, nhằm mục đích đưa các khuyến nghị cho việc giảm thiểu rủi ro của các lỗ hổng phần mềm.
 - Toàn văn tài liệu:

<https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-218.pdf>

<https://csrc.nist.gov/Projects/ssdf>

NIST Special Publication 800-218

Secure Software Development Framework (SSDF) Version 1.1:

Recommendations for Mitigating the Risk of Software Vulnerabilities

Murugiah Souppaya
Computer Security Division
Information Technology Laboratory

Karen Scarfone
Scarfone Cybersecurity
Clifton, VA

Donna Dodson*

* Former NIST employee; all work for this publication was done while at NIST.

This publication is available free of charge from:
<https://doi.org/10.6028/NIST.SP.800-218>

February 2022

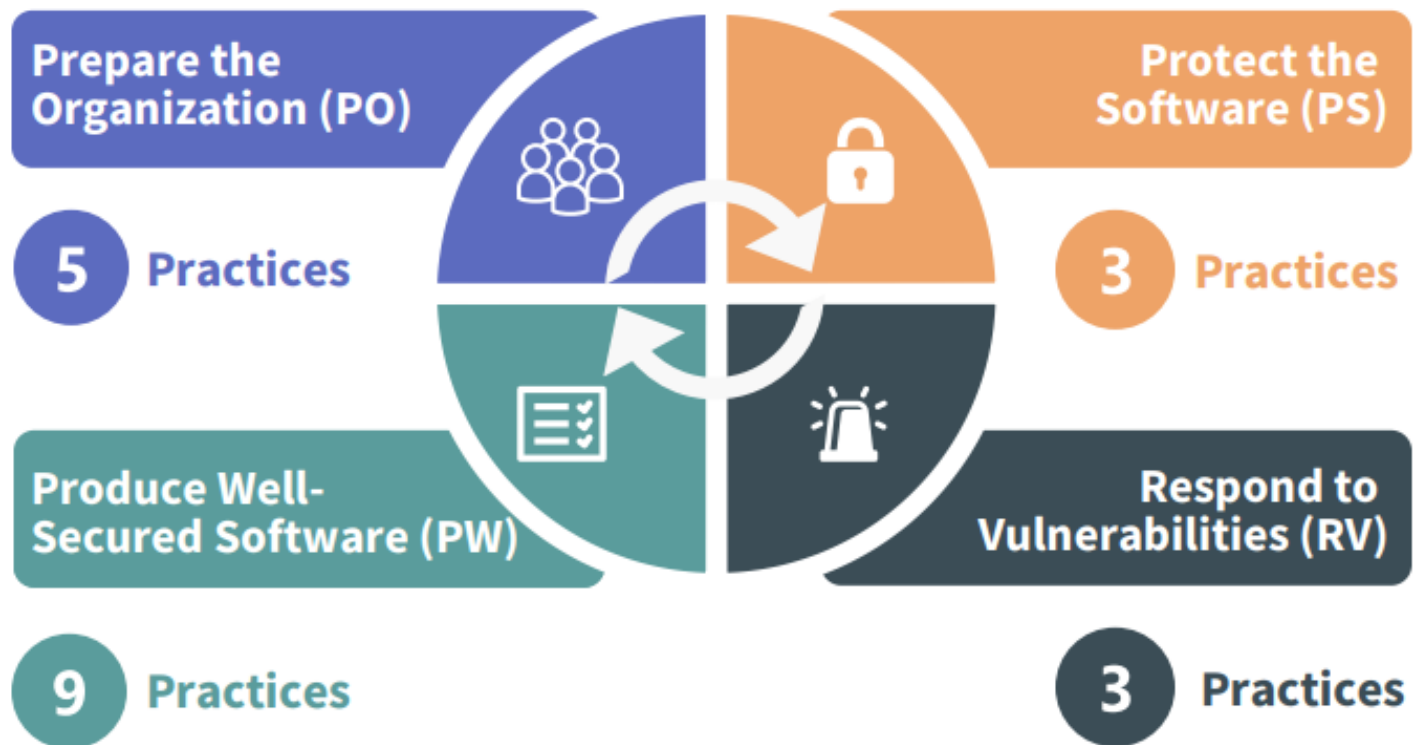


U.S. Department of Commerce
Gina M. Raimondo, Secretary

National Institute of Standards and Technology
James K. Olthoff, Performing the Non-Exclusive Functions and Duties of the Under Secretary of Commerce for Standards and Technology & Director, National Institute of Standards and Technology



Bộ khung phát triển phần mềm an toàn (2)



Prepare the Organization - Examples



Make sure the security requirements are defined and understood by your entire organization early. Update security requirements annually – at least. (PO 1.1)



Treat build systems like production systems by securing and hardening development endpoints. (PO 5.2)

Protect the Software - Examples



Use code signing to help protect the integrity of executables (PS 1.1)



Use an established certificate authority for verifying release integrity (PS 2.1)



Share provenance data e.g. in a software bill of materials [SBOM] (PS 3.2)

Produce Well Secured Software - Examples

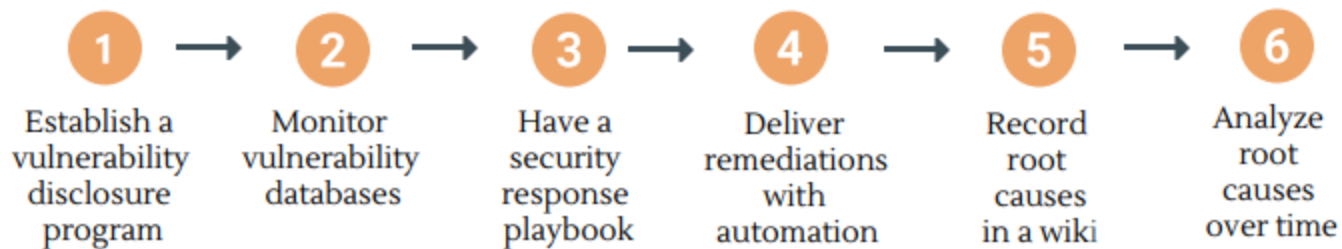


Reuse existing, well secured software (e.g. open source frameworks) instead of duplicating functionality. (PW 4.1)



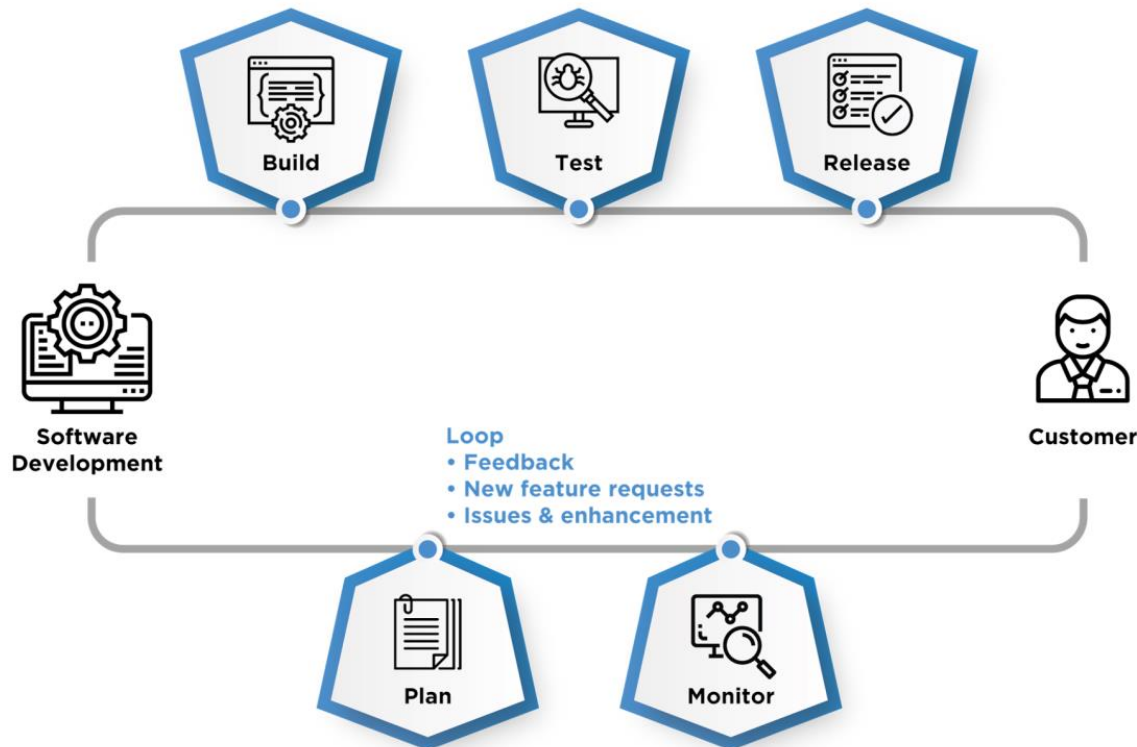
Implement "clean builds" and perform all builds in a dedicated, highly controlled build environment (PW 6.2)

Respond to Vulnerabilities - Examples



devops

- là **sự kết hợp** giữa **phát triển (development)** và **vận hành (operations)** phần mềm.
- Là văn hóa làm việc trong các công ty sử dụng quy trình phát triển Agile.

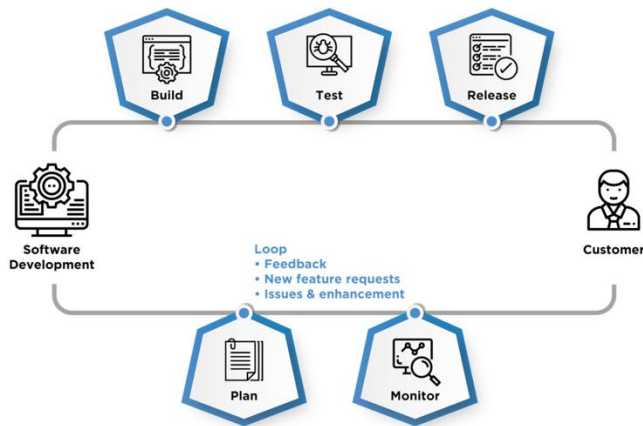


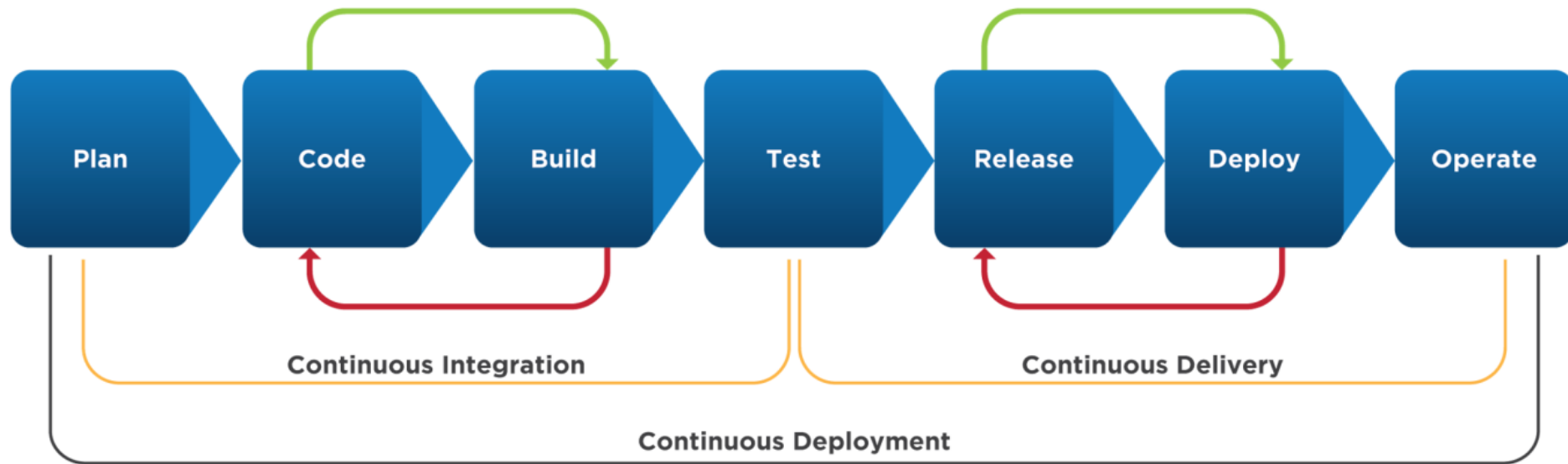
Phương pháp mới như **Continuous Integration, Continuous Delivery and Continuous Deployment** cùng với sự phát triển của DevOps, tập trung vào:

- Tương tác, phối hợp và gắn kết giữa các team.
- Áp dụng các phương pháp để tự động hóa thay đổi, cấu hình và triển khai.
- Chuyển giao giải pháp nhanh hơn.
- Theo dõi và lập kế hoạch cập nhật sản phẩm nhanh chóng.

Lợi ích của DevOps

- Cải thiện sự hợp tác, hỗ trợ chức năng và khắc phục lỗi nhanh hơn.
- Tăng tính linh hoạt, nhanh chóng và tin cậy.
- Bảo mật cơ sở hạ tầng và bảo vệ dữ liệu.
- Bảo trì và cập nhật nhanh hơn.
- Chuyển đổi các dự án với chiến lược số hóa.
- Tăng tốc độ, năng suất của team kinh doanh và CNTT.





Planning

- Requirement finalization
- Updates & new changes
- Architecture & design
- Task assignment
- Timeline finalization

Code

- Development
- Configuration finalization
- Check-in source code
- Static-code analysis
- Automated review & peer review

Build

- Compile code
- Unit testing
- Code-metrics
- Build container images or package
- Preparation or update in deployment templated
- Create or update monitor dashboards

Test

- Integration test with other component
- Load & stress test
- UI testing
- Penetration testing
- Requirement testing

Release

- Preparing release notes
- Version tagging
- Code freeze
- Feature freeze

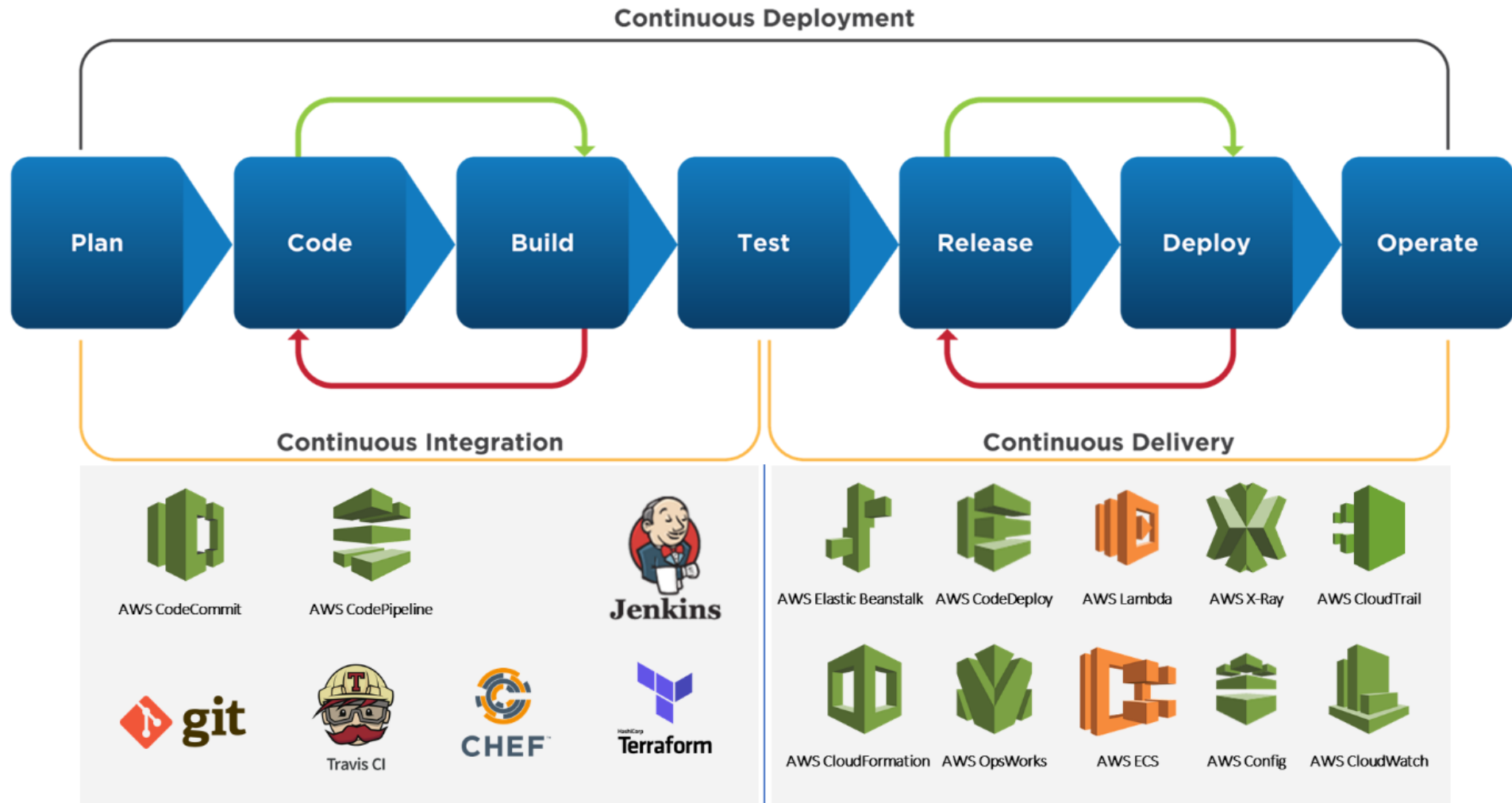
Deploy

- Updating the infrastructure i.e staging, production
- Verification on deployment i.e smoke tests

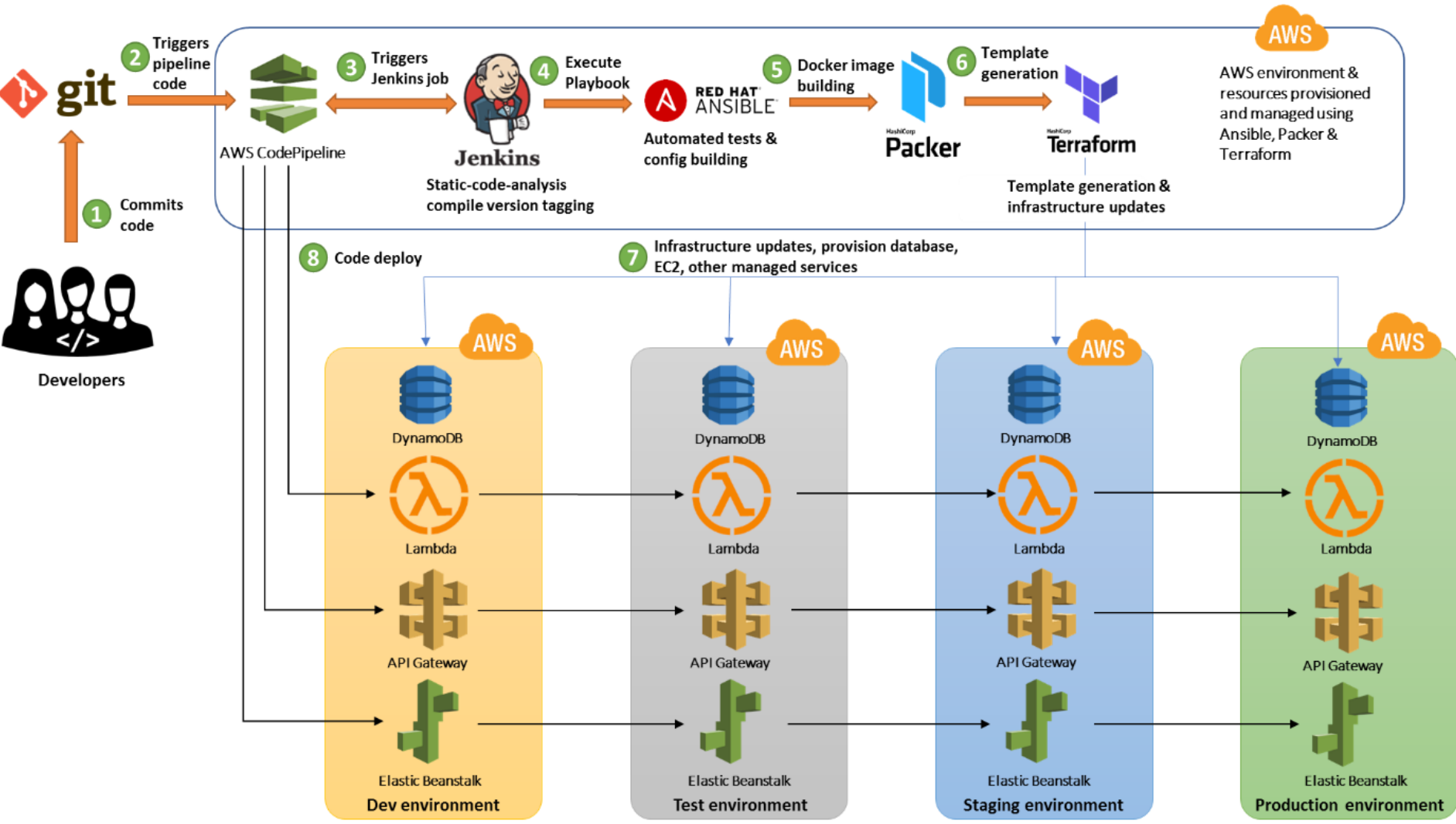
Operate

- Monitor designed dashboard
- Alarm triggers
- Automatic critical events handler
- Monitor error logs

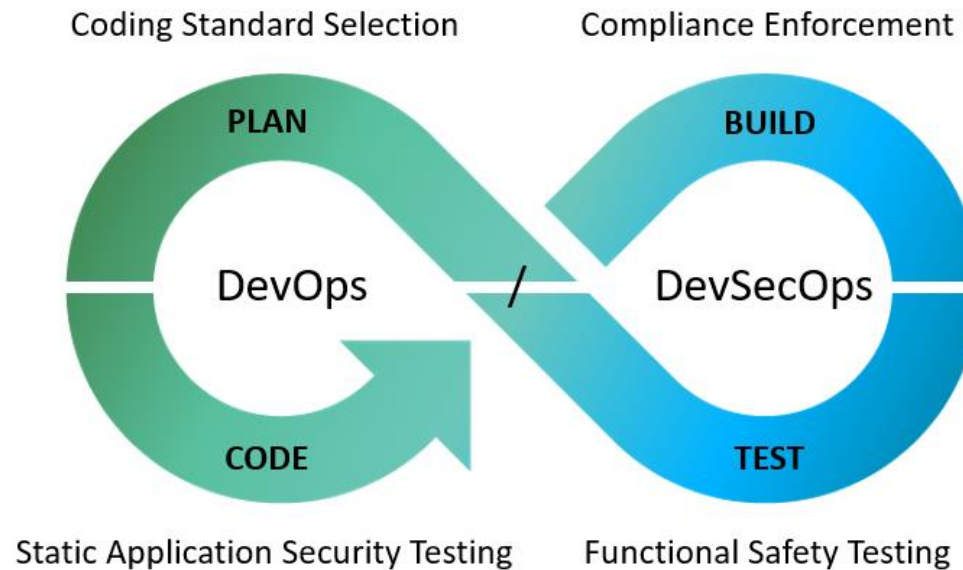
DevOps: Các công cụ



DevOps: Ví dụ



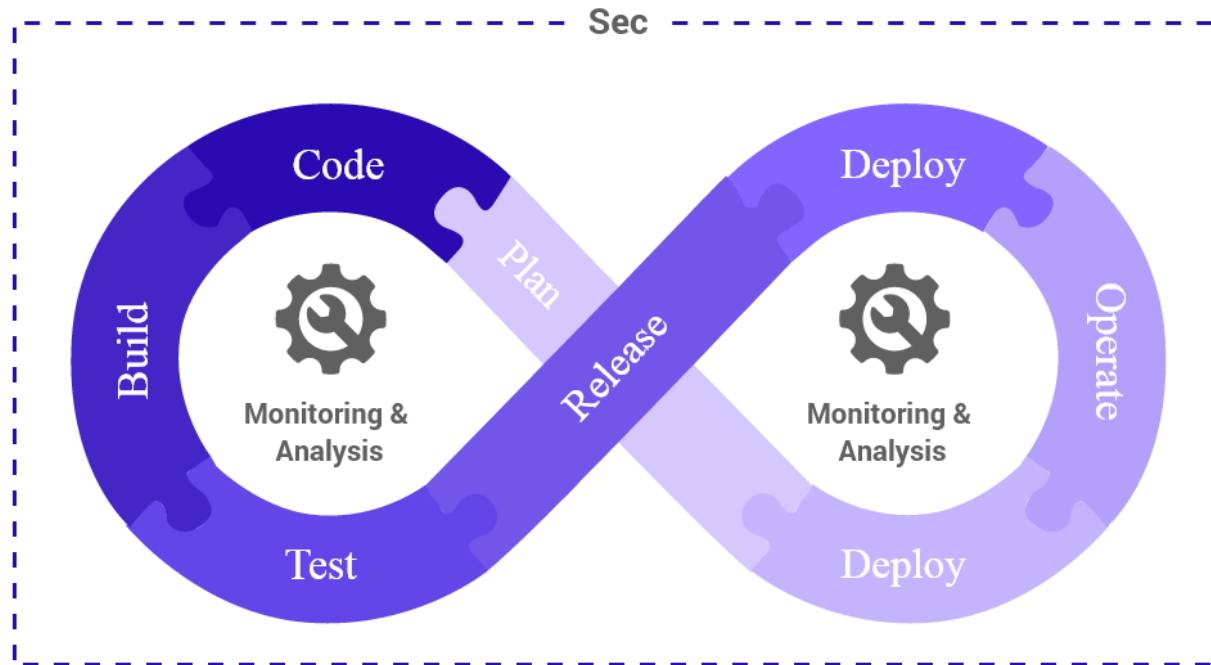
Lập trình an toàn với CI/CD



Continuous Integration (CI)
Continuous Delivery (CD)



DevSecOps: Triển khai Security vào pipeline

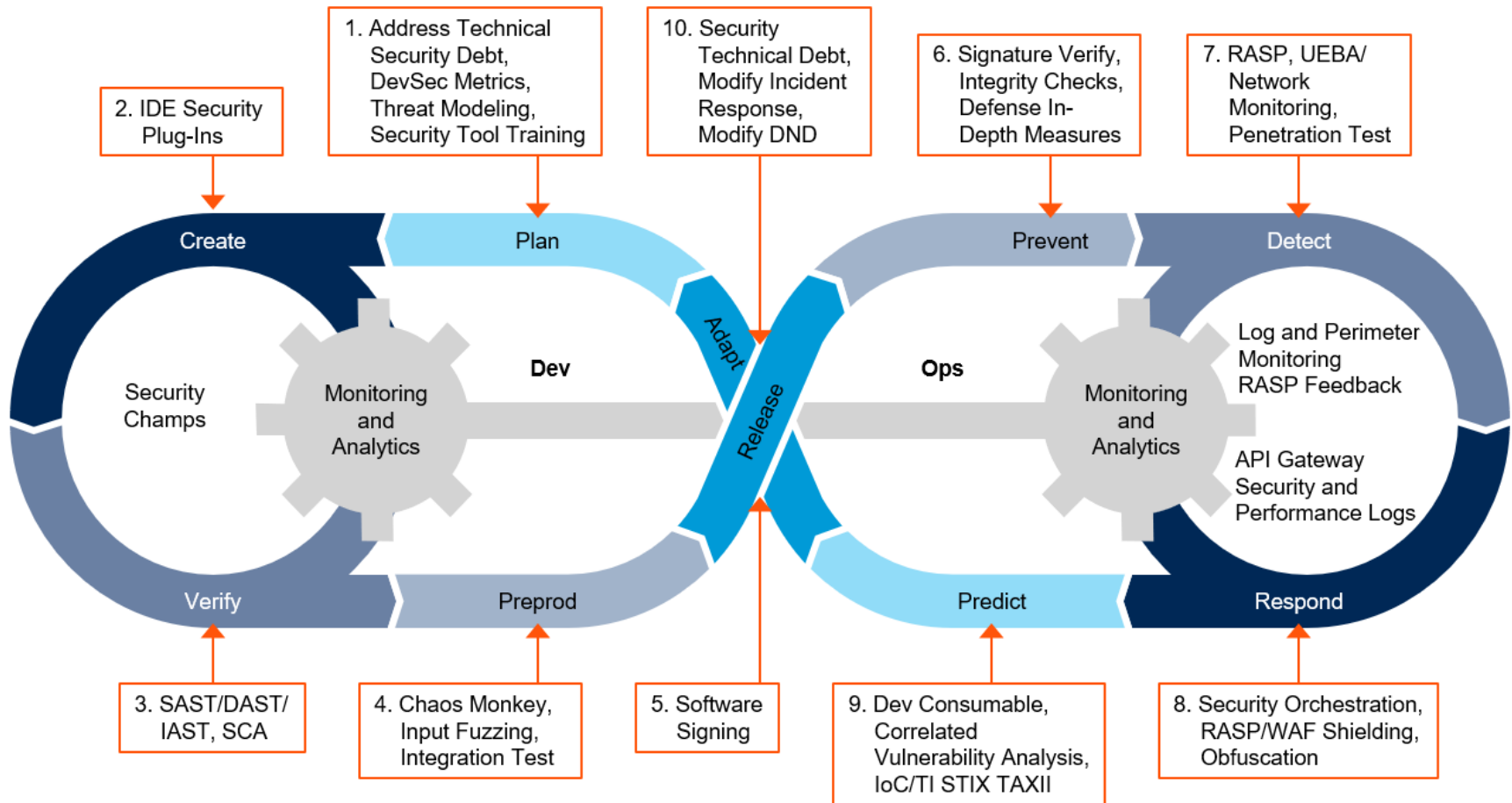


- Continuous Integration (CI)
- Continuous Delivery (CD)
- Continuous Inspection (CI)

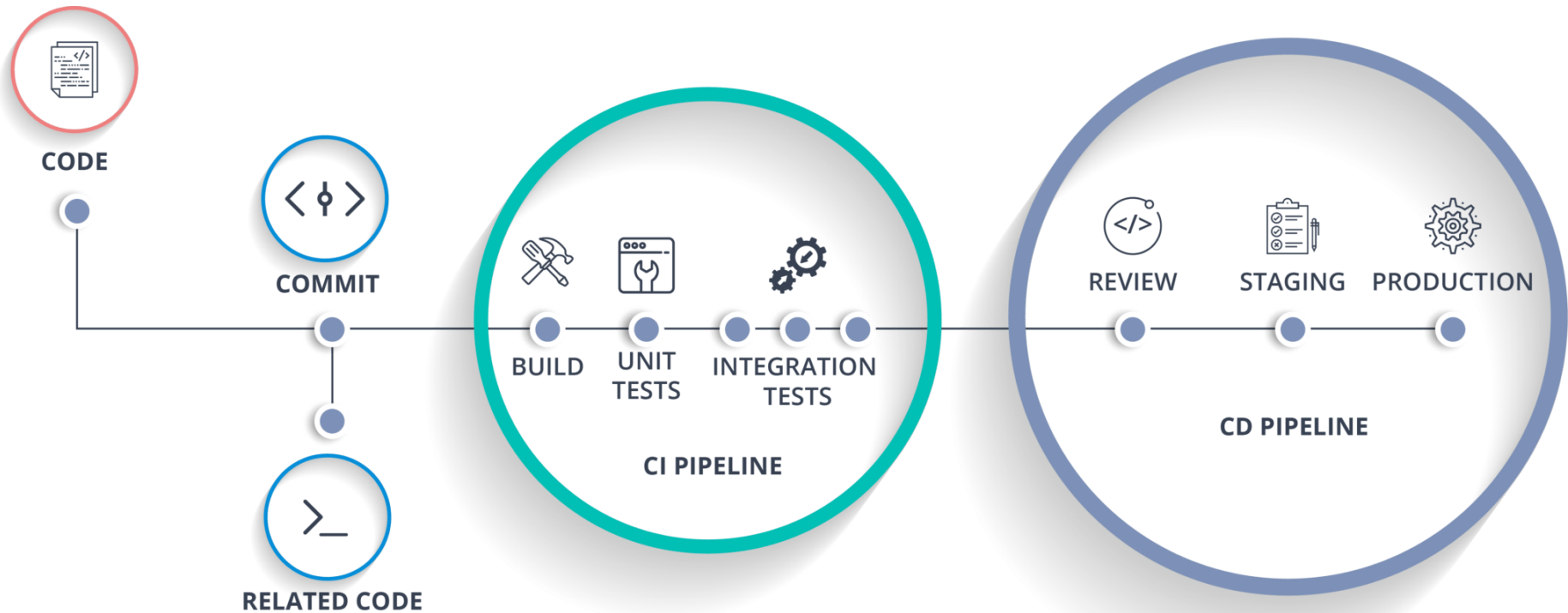


DevSecOps: Gartner Model

The DevSecOps Toolchain

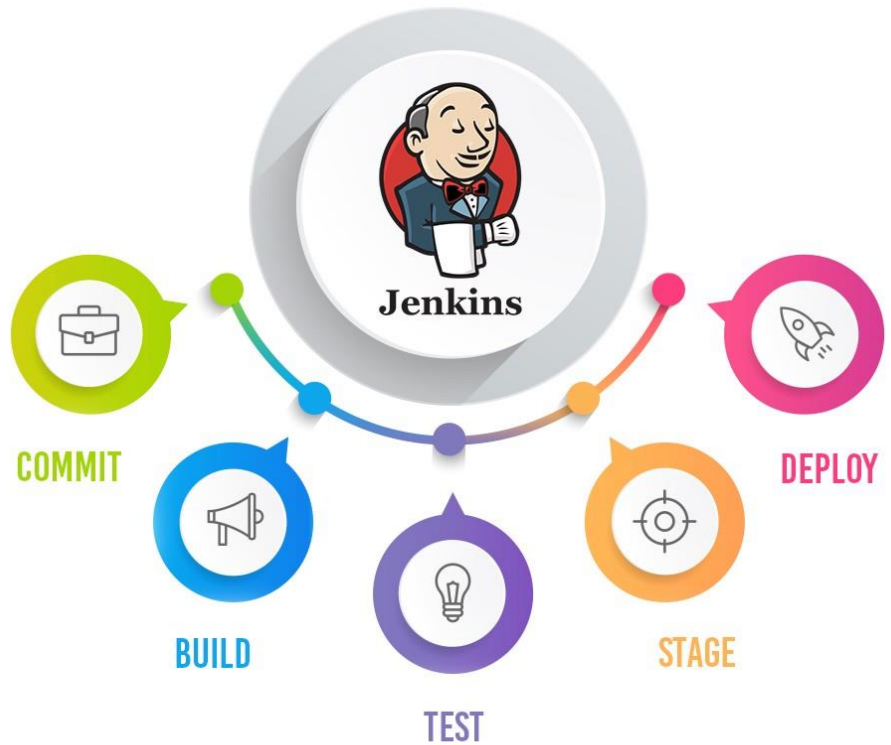
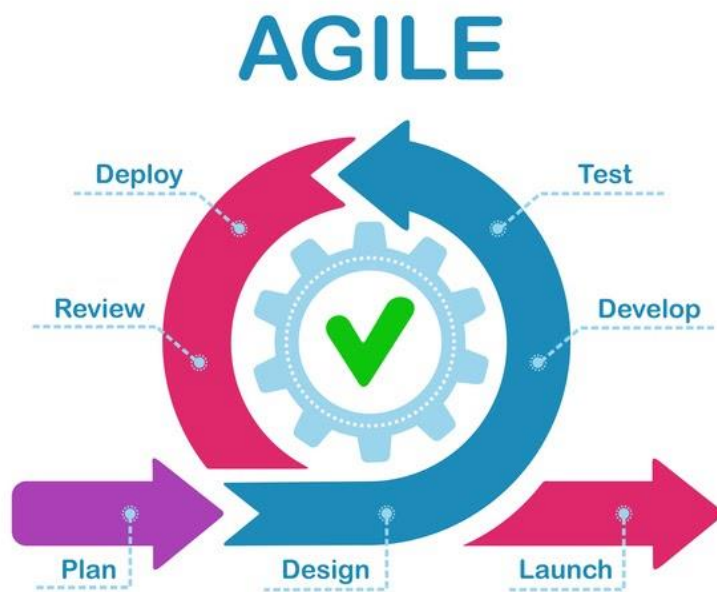


Lập trình an toàn với CI/CD

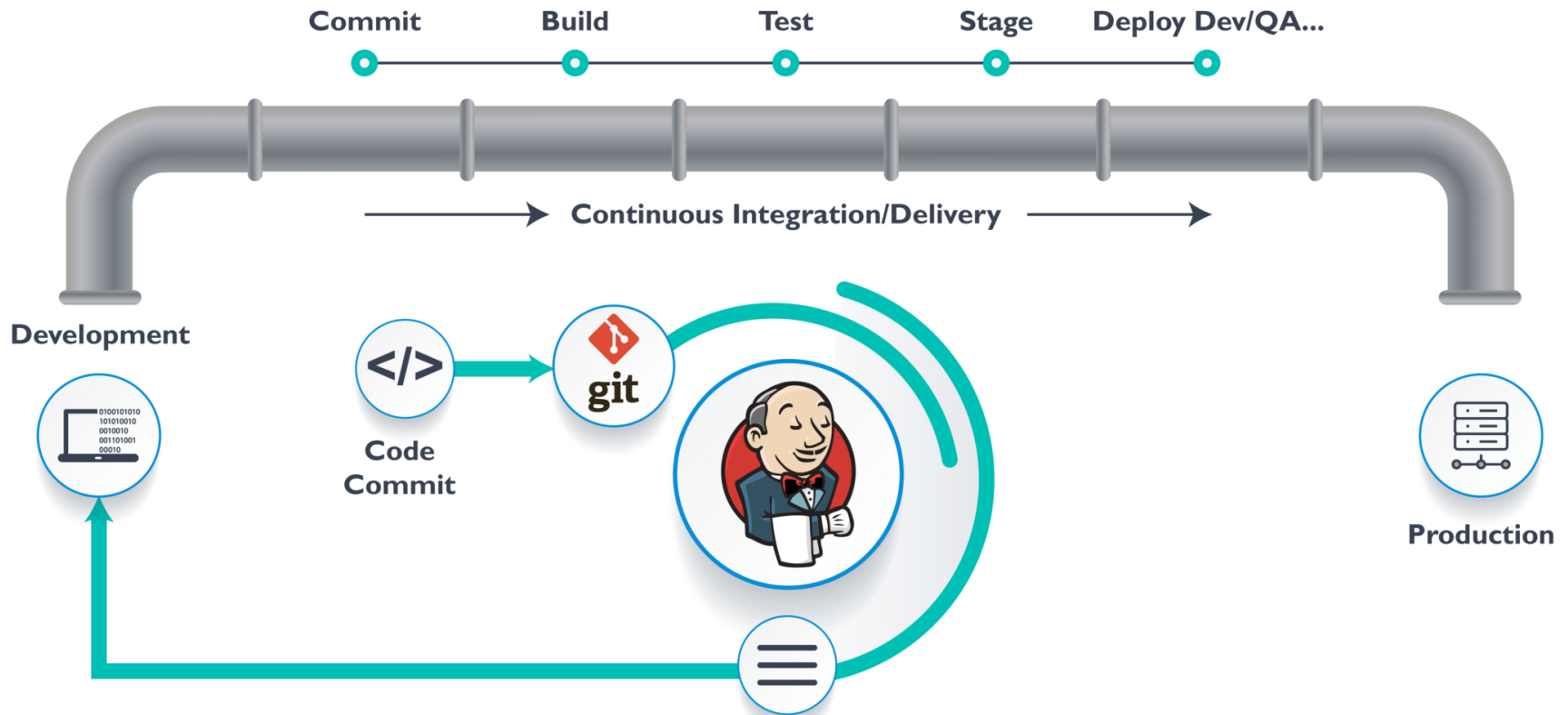


Continuous Integration (CI)
Continuous Delivery (CD)

Lập trình an toàn với CI/CD



Lập trình an toàn với CI/CD



Static Application Security Test (SAST)



Dynamic Application Security Test (DAST)



Static application security testing (SAST)

Giúp xác định các vấn đề trong mã nguồn thông qua tìm kiếm trong mã nguồn trước khi biên dịch.

Dynamic Application Security Testing (DAST):

Xác định các thông tin không thể tự động xác định từ bên trong mã nguồn.

Static Application Security Test (SAST)



Static application security testing (SAST)

Một số vấn đề có thể ngăn ngừa thông qua phân tích tĩnh:

- **Lộ tài nguyên:** phát hiện các tài nguyên bị lộ, có thể là lỗ hổng để tấn công.
- **Các khóa (keys) hay định danh (credentials) được code cứng.**
- **Vấn đề về sự ổn định:** ví dụ rò rỉ bộ nhớ như không giải phóng tài nguyên không còn được sử dụng, hoặc liên quan đến hiệu suất. Kẻ tấn công có thể cố gắng đánh sập hệ thống qua việc lợi dụng các vấn đề hiệu suất.

Dynamic Application Security Test (DAST)



- Quét các ứng dụng Web và phân tích các header được sử dụng.
- Tìm các điểm đầu cuối bị lộ và các chức năng bảo mật của chúng.
- Kiểm tra các giao tiếp/dịch vụ OpenAPI/SOAP/REST/Websocket/
- Kiểm tra lưu lượng mạng giữa trình duyệt và dịch vụ.

Application Dependency Vulnerability Scanning



Phân tích phụ thuộc

OWASP Dependency Check

- Có thể chạy từ Maven build và cung cấp dữ liệu cho Jenkins và SonarQube (bằng cách cài đặt plugin Dependency-Check).
- Ghi lại điểm CVSS (Common Vulnerability Scoring System) cho đánh giá rủi ro.

<https://owasp.org/www-project-dependency-check/>

Container Image Vulnerability Scanning



- **Anchore Engine** là dự án mã nguồn mở, cung cấp dịch vụ để kiểm tra, phân tích và chứng nhận các **image container**.
- **Anchore Engine** được cung cấp dưới dạng một image container Docker, có thể chạy độc lập hoặc trong một nền tảng điều hướng như Kubernetes, Docker Swarm, Rancher, Amazon ECS,...

Continuous Inspection



Continuous Inspection
sonarqube

<https://docs.sonarqube.org/latest/>

SonarQube® là một công cụ đánh giá mã nguồn tự động để phát hiện bug, lỗi hổng và “code smell” trong mã nguồn. Có thể tích hợp với luồng hoạt động đã có để kích hoạt CI (continuous inspection) trong các nhánh dự án và các yêu cầu pull.

For 27 languages >

Java >

JavaScript >

C# >

Python >

C++ >

COBOL >

58

Fortune
100 use
Enterprise
Editions

Products First

Our prime focus and dedication is in building great products that have an impact and are loved by their users.

IDE

sonarlint 

Fix Issues Before they Exist

[Discover SonarLint →](#)

CLOUD

sonarcloud 

Enhance Your Workflow with
Continuous Code Quality

[Discover SonarCloud ↗](#)

ON-PREMISE

sonarqube 

Your teammate for Code Quality and
Code Security

[Discover SonarQube ↗](#)

<https://www.sonarsource.com/>

Lập trình an toàn với CI/CD



Jenkins

Dashboard > Demo Pipeline > main > #26

[Back to Project](#)

Status

[Changes](#)

[Console Output](#)

[Edit Build Information](#)

[Delete build '#26'](#)

[Git Build Data](#)

[See Fingerprints](#)

[Test Result](#)

[Dependency-Check](#)

[SpotBugs Warnings](#)

[CPD Warnings](#)

[PMD Warnings](#)

[ZAP report](#)

[Maven](#)

[Anchore Report \(PASS\)](#)

[Restart from Stage](#)

[Replay](#)

[Pipeline Steps](#)

[Workspaces](#)

[Previous Build](#)

Build #26 (Mar 3, 2021 1:13:47 PM)

Build Artifacts

anchore_gates.json	3.23 KB view
anchore_security.json	13.92 KB view
anchoreengine-api-response-evaluation-1.json	17.34 KB view
anchoreengine-api-response-vulnerabilities-1.json	30.03 KB view
spring-boot-demo-0.0.1-SNAPSHOT.jar	26.39 MB view
spring-boot-demo-0.0.1-SNAPSHOT.pom	12.82 KB view

Changes

- Adding vulnerability ([details](#) / [githubweb](#))

Started by user [admin](#)

Revision: c6a94e4e3185527f9cd92b19df911ad42f666893

- main

Test Result (no failures)

SpotBugs: [One warning](#) ⓘ

- Reference build: [Demo Pipeline » main #25](#)

CPD: No warnings ⓘ

- No warnings for 16 builds, i.e. since build 11
- Reference build: [Demo Pipeline » main #25](#)

PMD: [One warning](#) ⓘ

- Reference build: [Demo Pipeline » main #25](#)

Anchore Report (PASS)



Jenkins: Building Java and deploying to Kubernetes

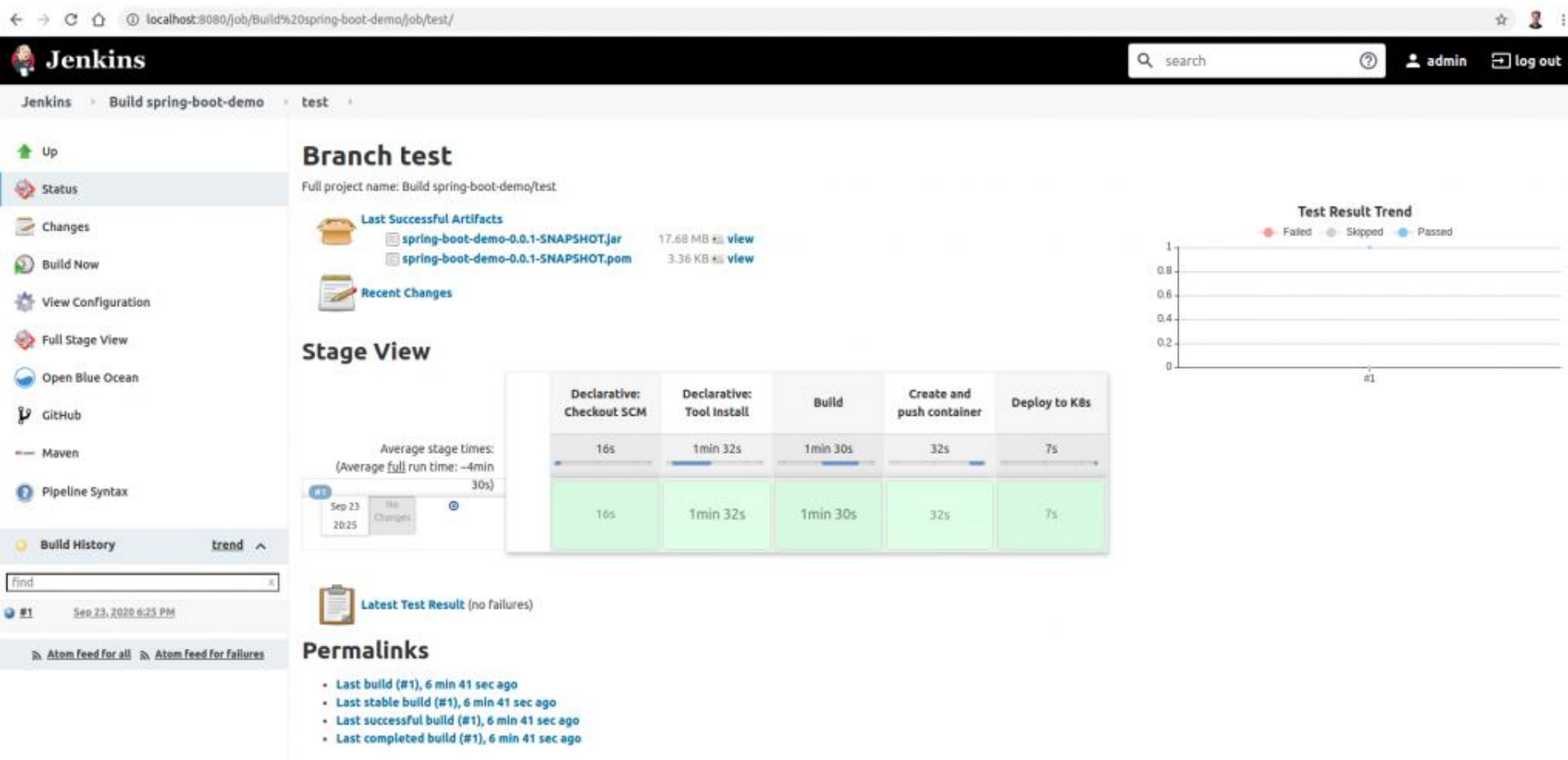
Kubernetes is a popular platform to run and manage containerized applications.

- Jenkins as CI/CD platform
- Kubernetes deployed with Kubespray on KVM ([here](#))
- MetalLB as Kubernetes loadbalancer
- GitHub as version control system
- Smee to forward Webhook calls from GitHub to Jenkins
- Maven to build the Java code
- Google Jib to wrap my compiled Java in a container
- DockerHub as container registry

Java Project: <https://github.com/MaartenSmeets/spring-boot-demo>

Lập trình an toàn với CI/CD

Jenkins: Building Java and deploying to Kubernetes



The screenshot displays the Jenkins web interface for a job named 'Branch test'. The interface includes a left sidebar with navigation options like 'Up', 'Status', 'Changes', 'Build Now', 'View Configuration', 'Full Stage View', 'Open Blue Ocean', 'GitHub', 'Maven', and 'Pipeline Syntax'. The main content area shows the 'Branch test' job details, including the full project name 'Build spring-boot-demo/test', 'Last Successful Artifacts' (spring-boot-demo-0.0.1-SNAPSHOT.jar and spring-boot-demo-0.0.1-SNAPSHOT.pom), and 'Recent Changes'. Below this is the 'Stage View' table, which shows the average stage times and the actual build times for the current build (#1).

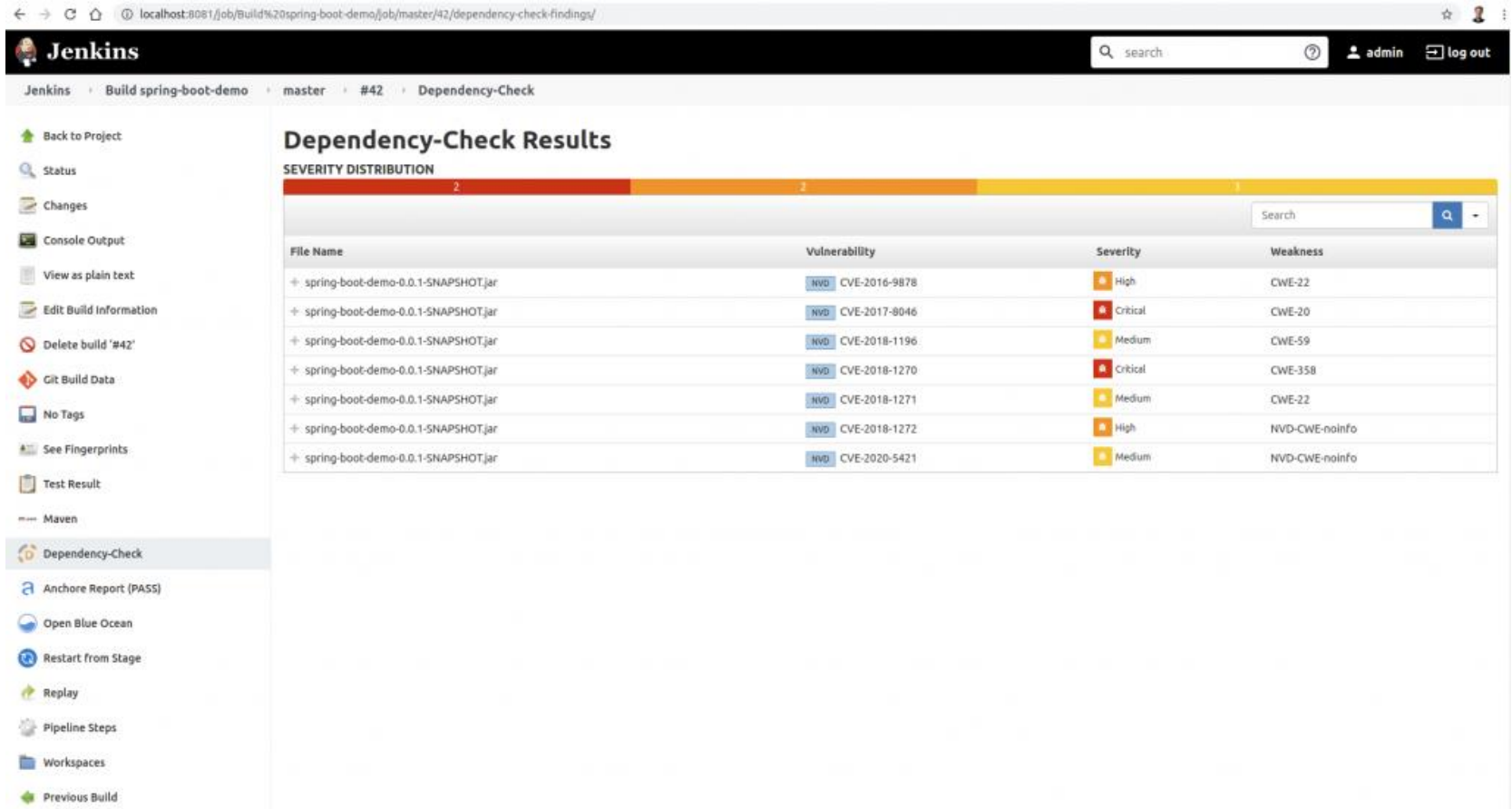
Stage	Declarative: Checkout SCM	Declarative: Tool Install	Build	Create and push container	Deploy to K8s
Average stage times:	16s	1min 32s	1min 30s	32s	7s
Actual build times (Build #1):	16s	1min 32s	1min 30s	32s	7s

The 'Test Result Trend' graph on the right shows a single data point for 'Passed' (blue dot) at build #1. The 'Latest Test Result' section indicates 'no failures'. The 'Permalinks' section provides links to the last build, last stable build, last successful build, and last completed build, all of which are the same build (#1).

Java Project: <https://github.com/MaartenSmeets/spring-boot-demo>

Lập trình an toàn với CI/CD

Jenkins Pipeline: SonarQube and the OWASP Dependency-Check on Kubernetes

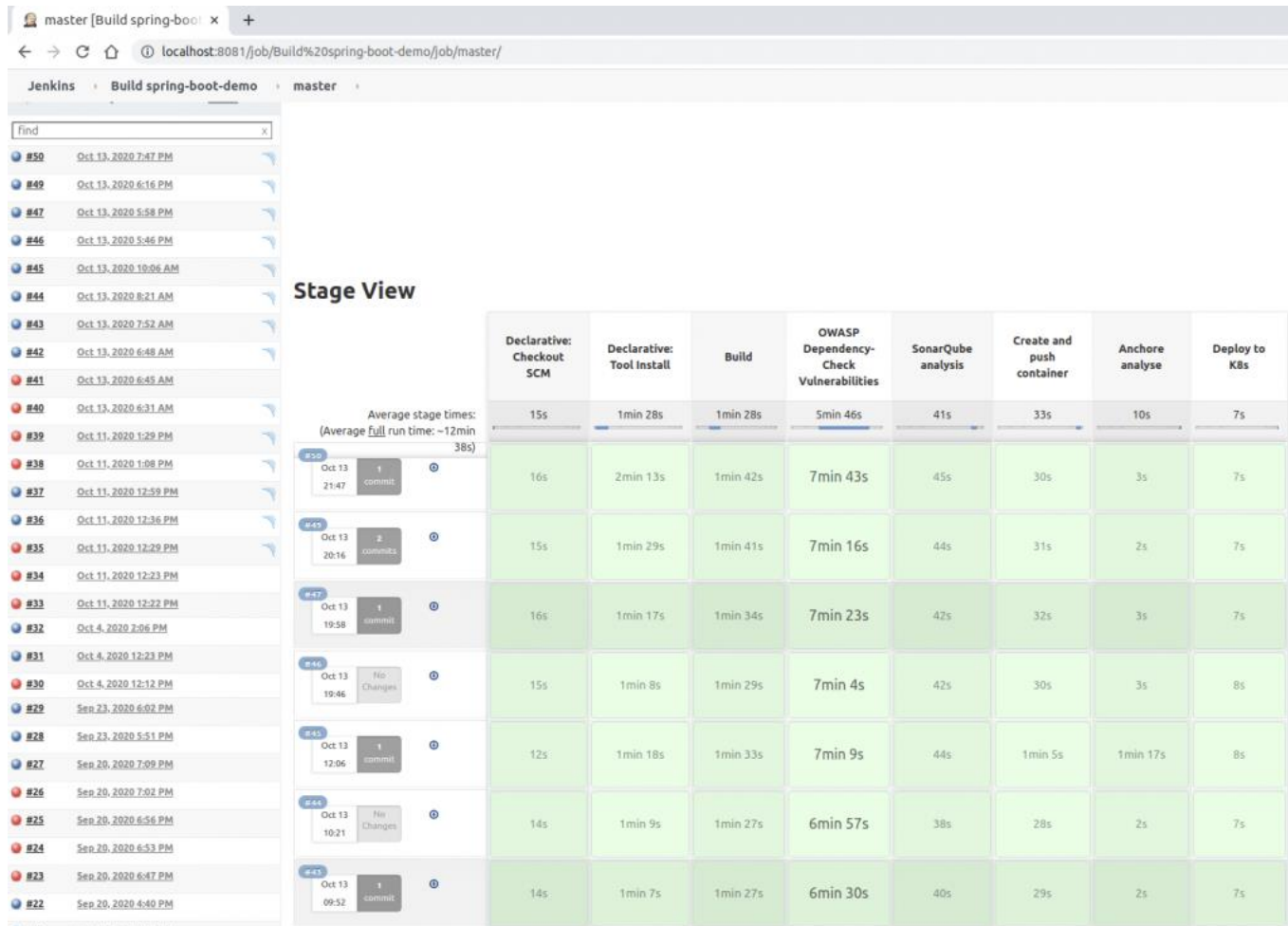


The screenshot shows the Jenkins web interface displaying the results of a Dependency-Check scan. The page title is "Dependency-Check Results". Below the title is a "SEVERITY DISTRIBUTION" bar chart showing 2 High, 2 Critical, and 1 Medium severity findings. A table lists the specific vulnerabilities found in the "spring-boot-demo-0.0.1-SNAPSHOT.jar" file.

File Name	Vulnerability	Severity	Weakness
+ spring-boot-demo-0.0.1-SNAPSHOT.jar	NVD CVE-2016-9878	High	CWE-22
+ spring-boot-demo-0.0.1-SNAPSHOT.jar	NVD CVE-2017-8046	Critical	CWE-20
+ spring-boot-demo-0.0.1-SNAPSHOT.jar	NVD CVE-2018-1196	Medium	CWE-59
+ spring-boot-demo-0.0.1-SNAPSHOT.jar	NVD CVE-2018-1270	Critical	CWE-358
+ spring-boot-demo-0.0.1-SNAPSHOT.jar	NVD CVE-2018-1271	Medium	CWE-22
+ spring-boot-demo-0.0.1-SNAPSHOT.jar	NVD CVE-2018-1272	High	NVD-CWE-noinfo
+ spring-boot-demo-0.0.1-SNAPSHOT.jar	NVD CVE-2020-5421	Medium	NVD-CWE-noinfo

Lập trình an toàn với CI/CD

Jenkins Pipeline: SonarQube and the OWASP Dependency-Check on Kubernetes





OWASP VulnerableApp

OWASP VulnerableApp Project: For Security Enthusiasts by Security Enthusiasts:

<https://github.com/SasanLabs/VulnerableApp>

Technology used:

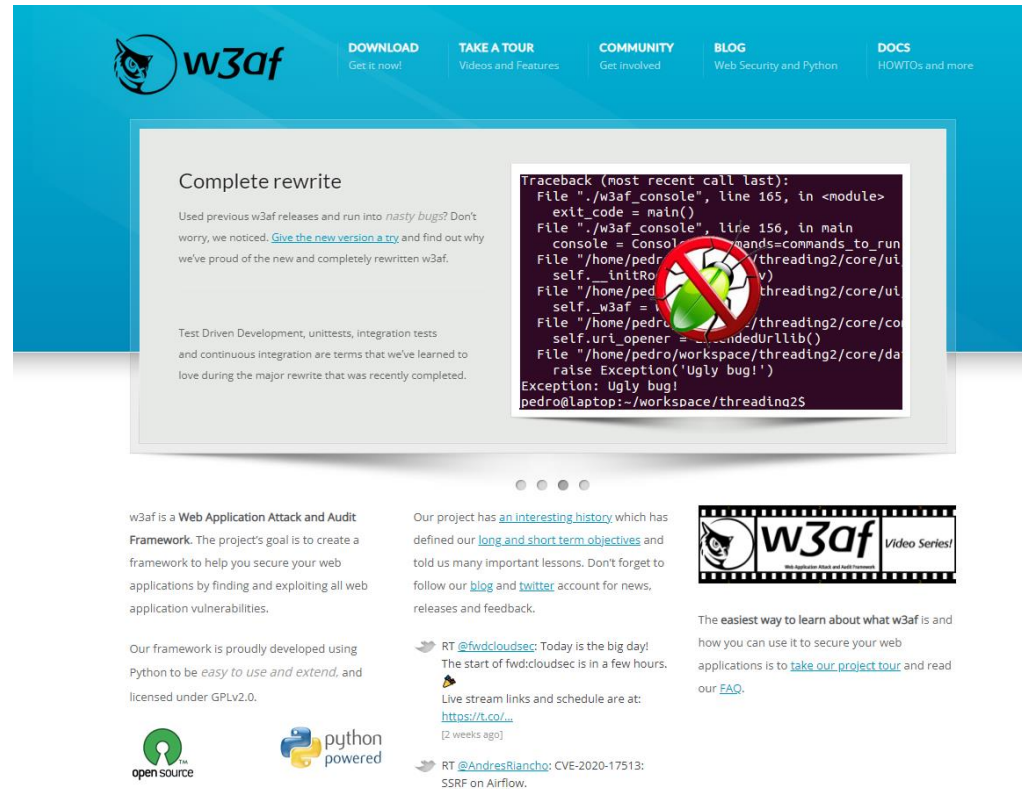
- Java8
- Spring Boot
- Vanilla Javascript



- Java Security Lab: <https://github.com/CSPF-Founder/JavaVulnerableLab>
- Appsecco: <https://github.com/appsecco>
- Damn Vulnerable Java (EE) Application: <https://github.com/appsecco/dvja>
- KubeSecO: Application Security Workflow Automation using Docker and Kubernetes
Link: <https://github.com/appsecco/kubeseco>



Lập trình an toàn với CI/CD



w3af

DOWNLOAD Get it now! TAKE A TOUR Videos and Features COMMUNITY Get involved BLOG Web Security and Python DOCS HOWTOs and more

Complete rewrite

Used previous w3af releases and run into *nasty bugs*? Don't worry, we noticed. [Give the new version a try](#) and find out why we've proud of the new and completely rewritten w3af.

Test Driven Development, unittests, integration tests and continuous integration are terms that we've learned to love during the major rewrite that was recently completed.

```
Traceback (most recent call last):
  File ".w3af_console", line 165, in <module>
    exit_code = main()
  File ".w3af_console", line 156, in main
    console = Console(commands=commands_to_run
  File "/home/pedro/workspace/threading2/core/ui
    self._initRouter()
  File "/home/pedro/workspace/threading2/core/ui
    self.w3af = W3af()
  File "/home/pedro/workspace/threading2/core/co
    self.uri_opener = urllib2.urlopen()
  File "/home/pedro/workspace/threading2/core/da
    raise Exception('Ugly bug!')
Exception: Ugly bug!
pedro@laptop:~/workspace/threading2$
```

w3af is a **Web Application Attack and Audit Framework**. The project's goal is to create a framework to help you secure your web applications by finding and exploiting all web application vulnerabilities.

Our framework is proudly developed using Python to be *easy to use and extend*, and licensed under GPLv2.0.

Our project has an [interesting history](#) which has defined our [long and short term objectives](#) and told us many important lessons. Don't forget to follow our [blog](#) and [twitter](#) account for news, releases and feedback.

RT @fwdcloudsec: Today is the big day! The start of fwdcloudsec is in a few hours. Live stream links and schedule are at: <https://t.co/...> [2 weeks ago]

RT @AndresRiancho: CVE-2020-17513: SSRF on Airflow.

The easiest way to learn about what w3af is and how you can use it to secure your web applications is to [take our project tour](#) and read our [FAQ](#).

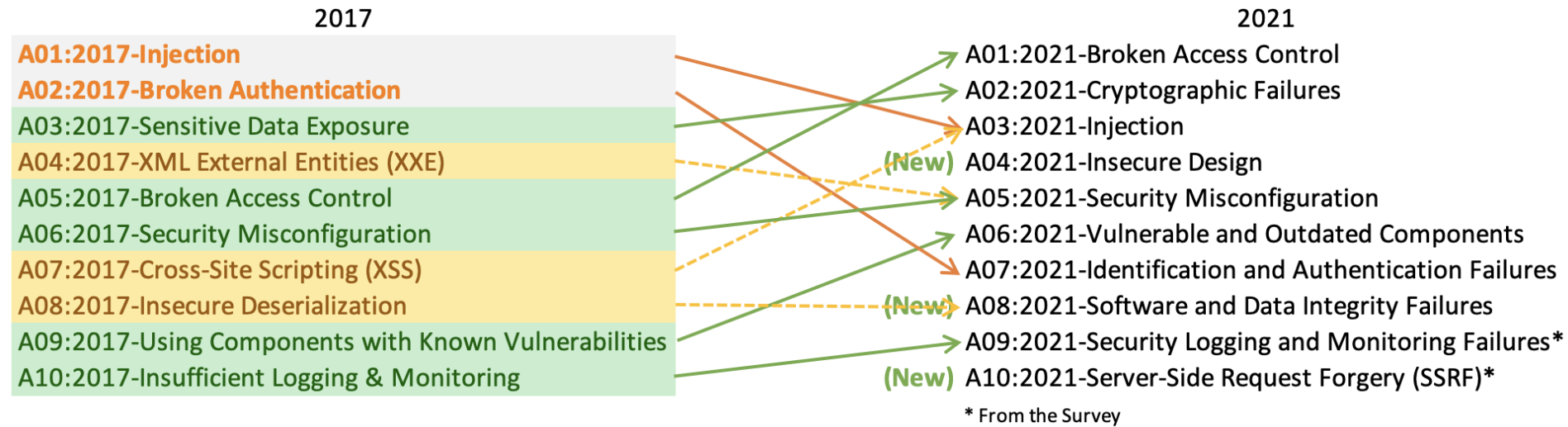
open source python powered

w3af Video Series!

w3af is a **Web Application Attack and Audit Framework**. The project's goal is to create a framework to help you secure your web applications by finding and exploiting all web application vulnerabilities.

Các lỗi thường gặp

- OWASP: Open Web Application Security Project, OWASP mobile security, ...



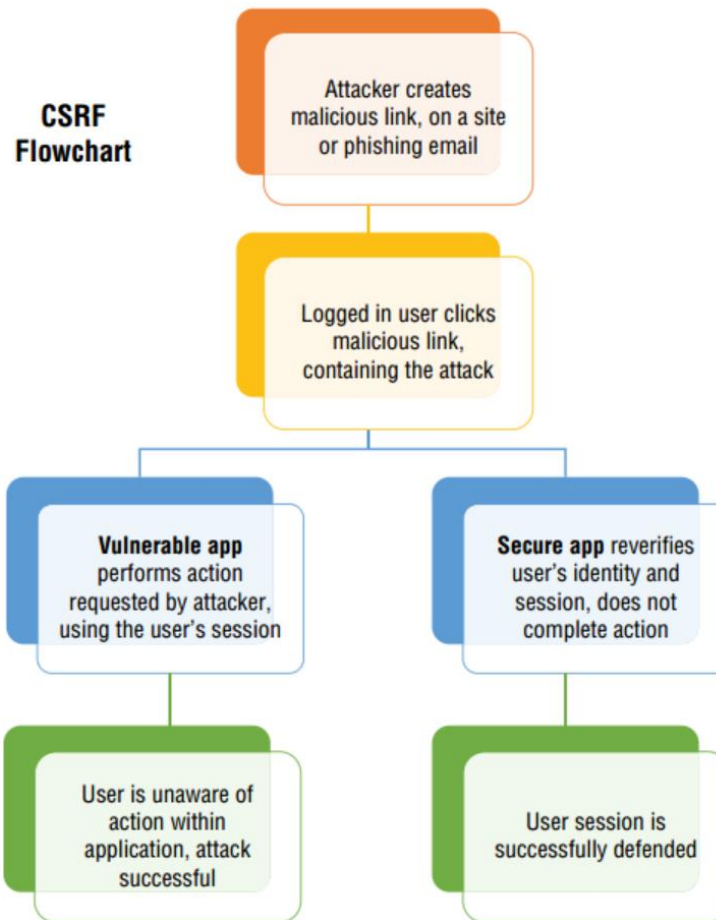


Figure 5-1: CSRF flowchart

Các lỗi thường gặp

Server-Side Request Forgery (SSRF)

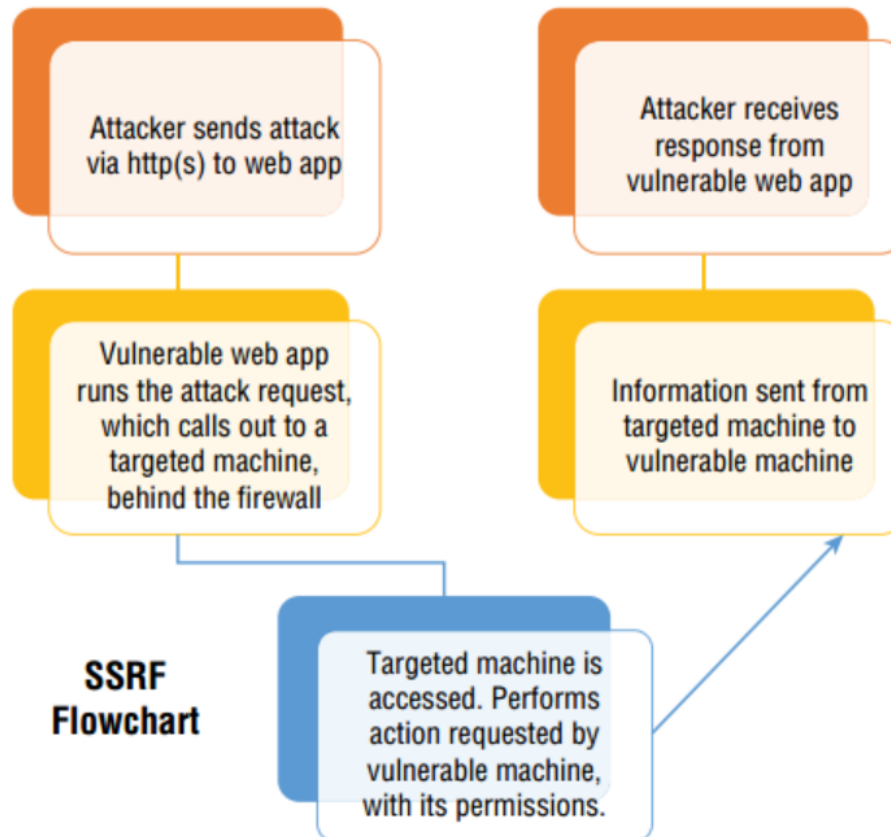


Figure 5-2: SSRF flowchart



Các lỗi thường gặp

Insecure Deserialization



- **Serialization vs Deserialization?**
 - **Serialization:** chuyển dữ liệu từ dạng object sang dạng có thể truyền hoặc lưu trữ trong stream hoặc file.
 - **Deserialization:** đưa dữ liệu về dạng object ban đầu sau khi truyền hoặc khi cần sử dụng.
- **Insecure Deserialization:** khi kẻ tấn công có thể thay thế dữ liệu đã serialize bằng dữ liệu độc hại nào đó, có thể gây ra hậu quả nghiêm trọng.
- Giải pháp: **hạn chế serialization/ deserialization khi có thể.** Nếu cần sử dụng object đã serialize, **chỉ sử dụng dữ liệu từ các nguồn tin cậy.**





- **Race condition** xảy ra khi các hoạt động của phần mềm thực thi dựa trên thời gian, và các tiểu trình cần được thiết lập đồng bộ, tuy nhiên không phải như vậy.

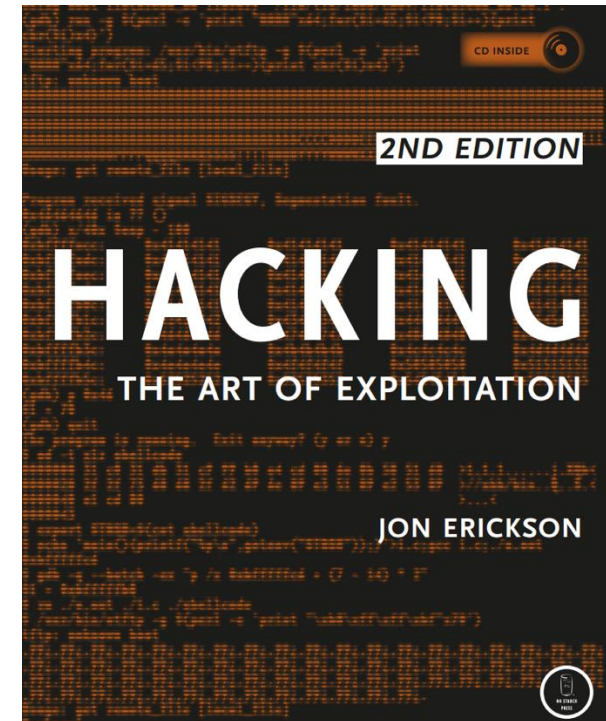
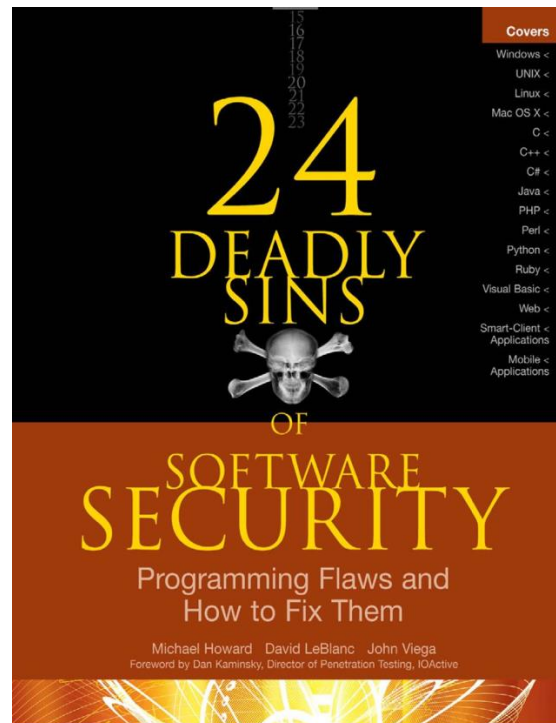
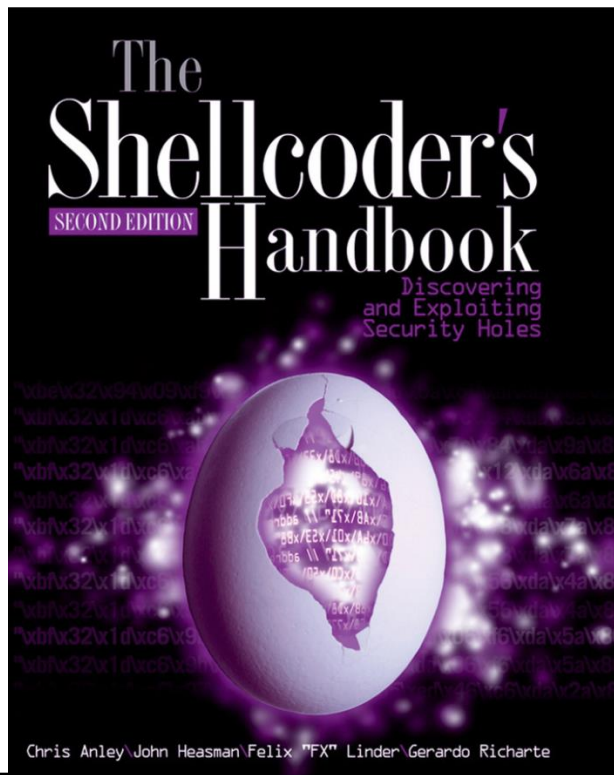


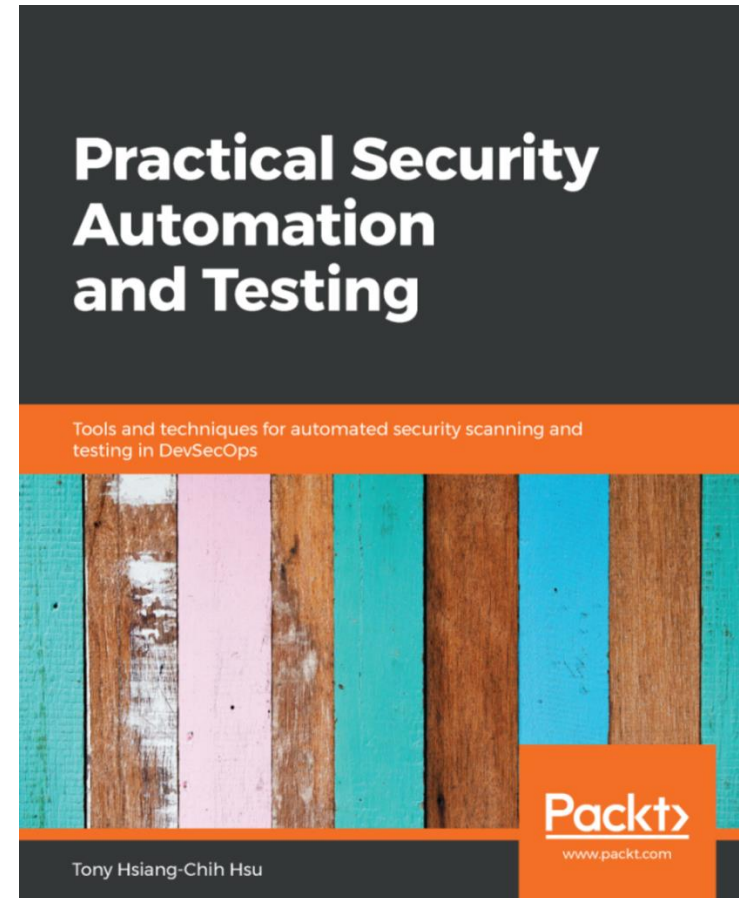
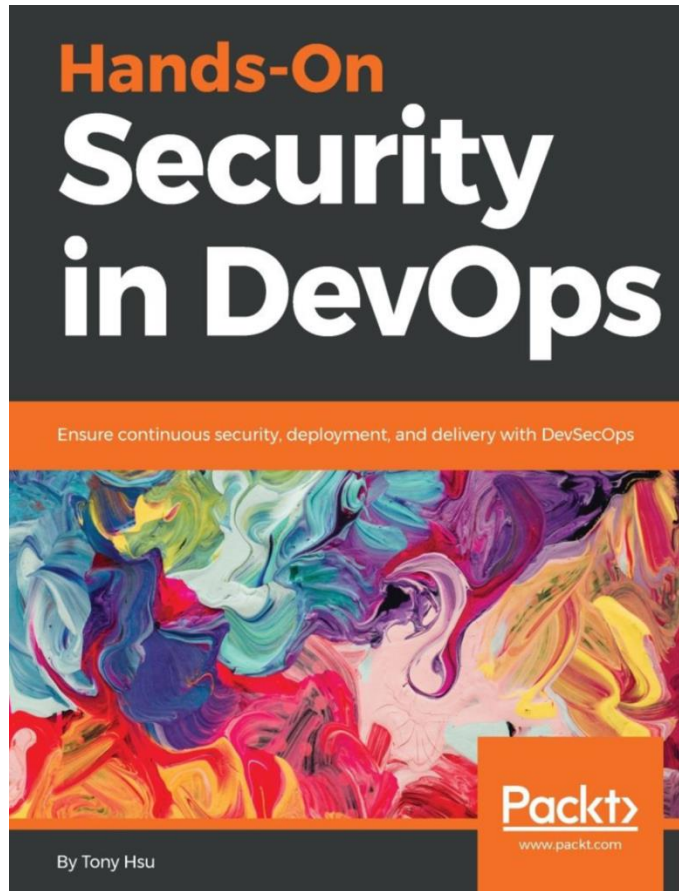


- Xem thêm trong OWASP, CWE,...



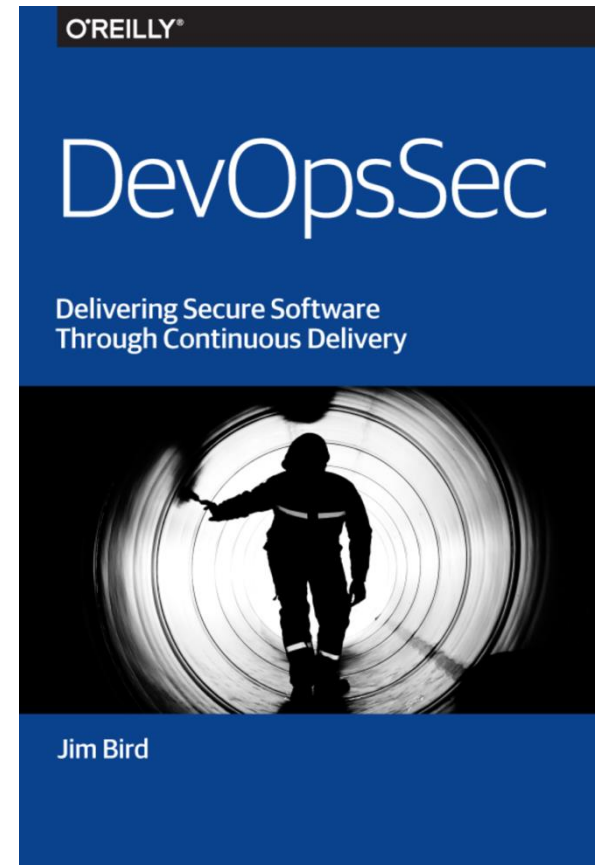
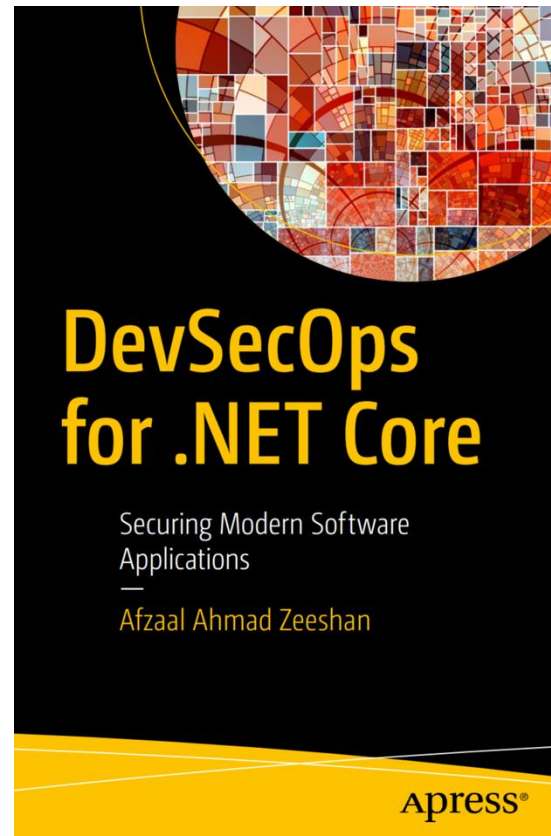
Tài liệu tham khảo

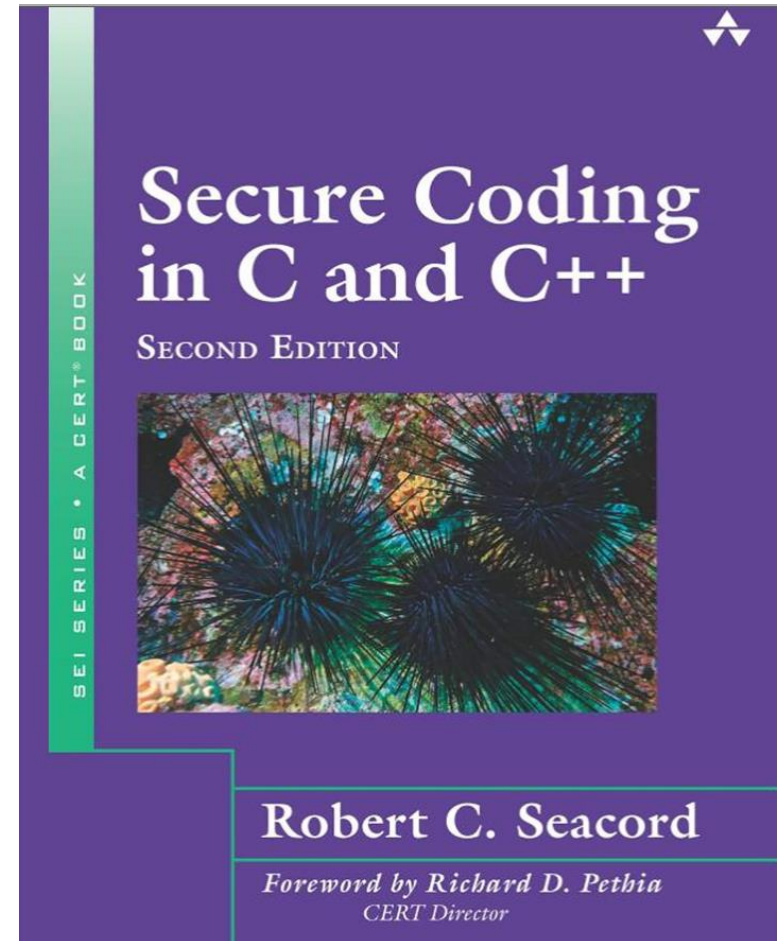
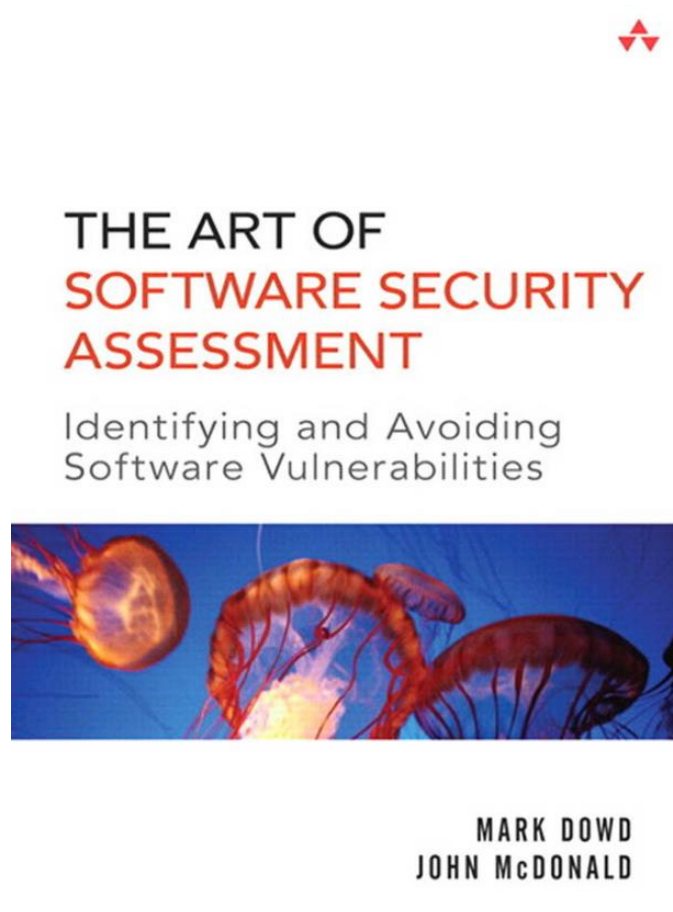


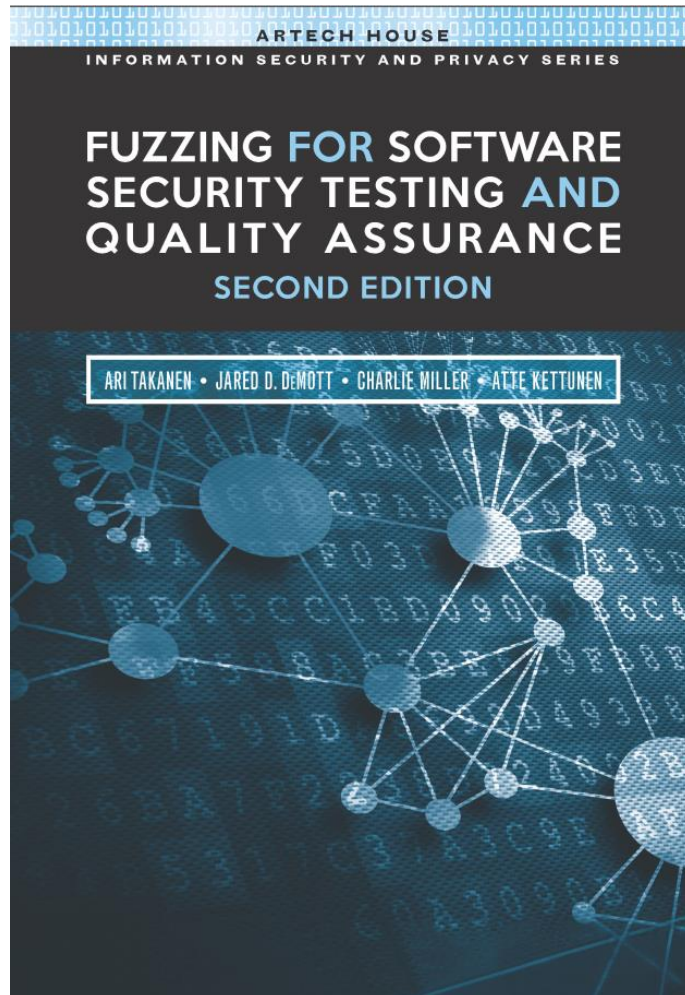




Laura Bell, Michael Brunton-Spall,
Rich Smith & Jim Bird









- <https://security.berkeley.edu/secure-coding-practice-guidelines>
- <https://wiki.sei.cmu.edu/confluence/display/sec+code/Top+10+Secure+Coding+Practices>
- https://owasp.org/www-pdf-archive/OWASP_SCP_Quick_Reference_Guide_v2.pdf
- <https://www.softwaretestinghelp.com/guidelines-for-secure-coding/>
- <http://security.cs.rpi.edu/courses/binexp-spring2015/>
- <https://www.ired.team/>

Lập trình an toàn & Khai thác lỗ hổng phần mềm



Trường ĐH CNTT TP. HCM